



University of Ioannina
Department of Computer Science and Engineering

Diploma Thesis

Force Control and Human Interaction for the FR3 Manipulator

Author: Danai Chanlaridou

Supervisor: Assistant Professor Konstantinos Vlachos

Ioannina, October 2026

Examining Committee

- Konstantinos Vlachos, Assistant Professor, Department of Computer Science and Engineering, University of Ioannina (Supervisor).
- Aristidis Lykas, Professor, Department of Computer Science and Engineering, University of Ioannina.
- Konstantinos Blekas, Professor, Department of Computer Science and Engineering, University of Ioannina.

Dedication

Special thanks to my family for supporting me throughout the years.

Acknowledgements

I would like to thank the University of Ioannina and, in particular, my supervisor, Assistant Professor Konstantinos Vlachos, for his guidance and support during the preparation of this thesis.

Abstract

This thesis investigates the implementation and evaluation of advanced interaction control strategies for the Franka Research 3 (FR3), a 7-degree-of-freedom (7-DOF) tactile robotic manipulator. While traditional industrial robots rely mainly on rigid position control, the development of human-robot collaboration (HRC) requires robots that are physically aware and capable of safe, compliant interaction.

The research exploits the integrated joint-torque sensors of the FR3 and the Franka Control Interface (FCI) in order to develop a torque-based impedance control framework. By modeling the manipulator as a virtual mass-spring-damper system, the robot becomes dynamically transparent and can respond to human-applied forces with high sensitivity. A central part of the work concerns the kinematic redundancy of the robot. By using null-space projection and differential kinematics, the system can execute secondary subtasks, such as obstacle avoidance and posture optimization, without disturbing the primary force-regulation task at the end-effector.

The theoretical framework is validated through simulations in Gazebo and real-world experiments. The results demonstrate the ability of the robot to maintain stable contact during surface-processing tasks while reacting safely to external human disturbances. Overall, the integration of high-bandwidth torque control with reactive redundancy resolution improves the safety and efficiency of bidirectional human-robot interaction in unstructured environments.

Extended Abstract

This diploma thesis focuses on the development and implementation of advanced interaction-control strategies for the Franka Research 3 robotic manipulator. In contrast to traditional industrial robots, which are usually based on rigid position control, safe human-robot collaboration requires systems with tactile awareness and compliant behavior. The FR3 was selected as the reference platform because it integrates torque sensors in all seven joints, offering high dynamic transparency and making it suitable for force-sensitive interaction.

The theoretical background of the thesis is based on differential kinematics and on the Elementary Transform Sequence (ETS) method for describing the kinematic chain. The Jacobian matrix, denoted by J , plays a central role because it maps task-space velocities to joint velocities and, through force-velocity duality, maps end-effector forces to joint torques:

$$\tau = J^T F. \quad (1)$$

Special emphasis is placed on the redundancy of the 7-DOF manipulator. Through null-space projection,

$$N = I - J^\dagger J, \quad (2)$$

secondary tasks such as obstacle avoidance and posture optimization can be executed without affecting the primary force-control task at the end-effector.

The thesis implements a torque-based impedance controller. The manipulator is modeled as a virtual mass-spring-damper system, which enables programmable compliance during contact. The Franka Control Interface is used to send commands at a frequency of 1 kHz, ensuring the stability and response speed required for smooth interaction with humans.

The system is evaluated through Gazebo simulations and experimental tests on real hardware. The results show that the combination of impedance control and redundancy management allows the FR3 to perform precision tasks, such as surface processing, while responding safely to external human interventions. The work also shows that the use of the FR3's internal torque sensors can reduce the need for expensive external wrist force/torque sensors, decreasing both the cost and the complexity of collaborative robotic applications.

Contents

Examining Committee	i
Dedication	ii
Acknowledgements	iii
Abstract	iv
Extended Abstract	v
1 Introduction	1
1.1 Motivation	1
1.2 State of the Art of Control Algorithms	1
1.3 Goals	2
2 Mathematical Model	4
2.1 Robot Configuration	4
2.2 Forward and Inverse Kinematics	6
2.3 The Jacobian Matrix	8
2.4 Redundancy and Null-Space Projection	11
2.5 Singularities	13
2.6 Joint Space and Operational Space	17
2.7 Dynamic Analysis	17
2.7.1 Equations of Motion	18
2.7.2 Forward Dynamics and Inverse Dynamics	20
2.8 Force Sensors	21
2.9 Wrench Force Sensors	23
3 Force Control and Sensing	27
3.1 Manipulator Interaction with the Environment	27
3.2 Force Control	28
3.3 Physical and Artificial Constraints	28
3.4 Compliance Control	28

3.5	Impedance Control	29
3.6	Admittance Control	32
3.7	Hybrid Force/Position Control	34
3.8	Comparison of the Controllers	37
3.9	Pseudo-Algorithms of the Implemented Controllers	38
3.9.1	Pseudo-Algorithm of the Cartesian Impedance Controller	38
3.9.2	Pseudo-Algorithm of the Cartesian Admittance Controller	40
3.9.3	Pseudo-Algorithm of the Hybrid Force/Position Controller	42
3.9.4	Comparison of the Three Algorithms	46
4	Hardware/Technical specifications	47
4.1	Hardware and Technical Specifications	47
4.1.1	FR3 Arm Hardware Overview	49
4.1.2	Joint Position Limits	51
4.1.3	Joint Velocity and Cartesian Speed Limits	52
4.1.4	Control and Interaction Performance	53
4.1.5	Franka Hand Technical Specifications	55
4.1.6	Payload and End-Effector Configuration	57
4.1.7	Mounting and Installation Requirements	58
4.1.8	Network and FCI Configuration	59
4.1.9	Safety Functions and Practical Limits	60
4.1.10	Importance of Technical Limits for the Controllers	61
5	Simulation Results	62
5.1	The Robot Operating System	62
5.2	Gazebo Simulator	63
6	Experimental Results	66
6.1	Franka Control Interface	66
6.1.1	Experimental Campaign	68
6.1.2	Experiment 1: Baseline Free-Space Joint Trajectory	69
6.1.3	Experiment 2: Contact Approach with a Soft Object	69
6.1.4	Experiment 3: Constant Force Pressing Experiment	70
6.1.5	Experiment 4: Human Push and Disturbance Rejection	71
6.1.6	Experiment 5: Hand-Guiding and Lead-Through Experiment	72
6.1.7	Experiment 6: Pick and Place Demonstration	72
6.1.8	Experiment 7: Variable Stiffness Impedance Experiment	73
6.1.9	Experiment 8: Physiotherapy Test Experiment	74
6.1.10	Experiment 9: Different Contact Materials	74
6.1.11	Summary of the FCI Experiments	75

7	Conclusions and Future Work	76
7.1	Conclusions	76
7.2	Challenges and Limitations	76
7.3	Future Work	77
8	Bibliography	78

Chapter 1

Introduction

1.1 Motivation

The field of industrial robotics is undergoing a significant paradigm shift. For many decades, robotic manipulators were designed as rigid, high-precision machines operating in structured environments and isolated from humans by safety fences. However, the modern vision of Industry 4.0 and human-robot collaboration requires a transition toward tactile robotics, where robots can safely share their workspace with people.

The primary motivation of this research is to enable robots to operate in unstructured spaces where human contact is not necessarily an error, but can be part of the task. Traditional position-controlled robots do not provide the dynamic transparency required for safe physical interaction. This thesis is therefore motivated by the need for compliant control systems that can sense environmental forces and respond with human-like sensitivity, ensuring safety without compromising task efficiency.

1.2 State of the Art of Control Algorithms

The state of the art in robotics is defined by the transition from rigid automation to physically aware autonomy. Stable and sensitive physical interaction with the environment is one of the defining characteristics of advanced manipulation. A robot that can regulate its applied force and adapt its apparent compliance can operate more safely and effectively in collaborative and uncertain environments.

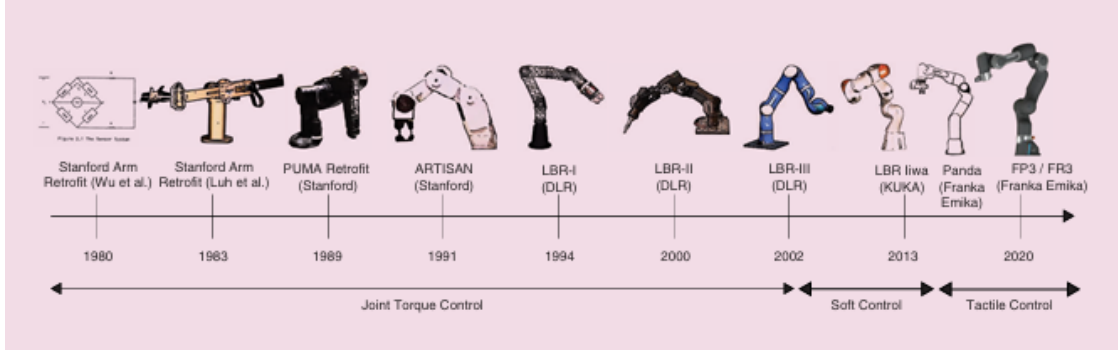


Figure 1.1: Important development steps in joint-torque control, from early theoretical foundations to modern product implementations.

Figure 1.1 depicts important development steps in joint-torque control, beginning with early theoretical work and reaching modern product implementations. These developments laid the foundations for soft and tactile control in advanced industrial robots. The mathematical framework for controlling the interaction between the FR3 and its environment can be categorized according to how the controller handles the relationship between motion and force.

1.3 Goals

The main goal of this thesis is to study, implement, and experimentally evaluate force-control methods for the Franka Research 3 manipulator, with emphasis on safe physical interaction between the robot, the environment, and a human operator. The work focuses not only on the theoretical design of force-control algorithms, but also on the practical implementation of a complete experimental workflow using the Franka Control Interface, `libfranka`, ROS 2, MoveIt, Gazebo, and real robot experiments.

The specific objectives of this thesis are:

1. **Study of the FR3 platform and its force-control capabilities:** to understand the kinematic, dynamic, sensing, and control characteristics of the Franka Research 3, especially its seven-degree-of-freedom structure, integrated joint torque sensing, external wrench estimation, and torque-level control through the Franka Control Interface.
2. **Implementation of direct real-time control using `libfranka`:** to establish direct communication with the real FR3 without ROS, read the robot state, access force and torque estimates, send control commands, and execute safe real-time experiments through the Franka Control Interface.
3. **Development and testing of force-control strategies:** to implement and compare the main interaction-control approaches used in this thesis, namely

Cartesian impedance control, Cartesian admittance control, and hybrid force/position control.

4. **Evaluation of compliant human–robot interaction:** to investigate how the FR3 behaves during physical interaction, including soft contact, external human pushes, hand-guiding behavior, and variable stiffness response.
5. **Simulation and visualization using ROS 2, MoveIt, RViz, and Gazebo:** to create a safe development workflow where robot models, controllers, planning groups, and simulation scenes can be tested before executing experiments on the real robot.
6. **Execution of real robot experiments:** to perform a set of practical experiments, including free-space joint motion, contact approach with soft objects, constant force pressing, human disturbance response, hand-guiding, pick-and-place motion, variable stiffness impedance, physiotherapy-inspired interaction, and contact with different materials.

Overall, the thesis aims to demonstrate that the FR3 can be used as a force-aware collaborative robot platform for safe contact tasks and human–robot interaction. The final objective is not only to design controllers theoretically, but also to validate them through a complete practical pipeline from simulation to real hardware execution.

Chapter 2

Mathematical Model

2.1 Robot Configuration

To address the challenges of force-sensitive interaction, this research uses the Franka Research 3, a state-of-the-art 7-DOF tactile robot platform. Unlike conventional collaborative robots that estimate forces only through motor currents, the FR3 is equipped with integrated strain-gauge torque sensors in all seven joints.

This hardware architecture provides the following advantages:

- **Redundancy:** the 7-DOF anthropomorphic configuration provides an internal degree of freedom, or null space, which is essential for obstacle avoidance and posture optimization.
- **High-bandwidth control:** through the Franka Control Interface, the robot supports a 1 kHz control loop, which is a prerequisite for stable and smooth impedance control.
- **Force sensitivity:** the ability to sense and command torques directly allows the FR3 to perform delicate tasks that are difficult for purely position-controlled manipulators.

Dynamic transparency describes the ability of the FR3 to compensate for its own weight, friction, and dynamic effects so that interaction with a human user feels as natural as possible. In this thesis, the term unstructured environment refers to a workspace that can contain humans, objects, obstacles, and uncertain contact conditions.

A precise kinematic model is required in order to control the FR3. The Denavit-Hartenberg convention has traditionally been used for modeling serial robotic chains. However, this thesis adopts the Elementary Transform Sequence (ETS) representation. Unlike Denavit-Hartenberg parameters, which can be counter-intuitive and constrained by specific frame alignments, ETS describes the robot as a sequence of elementary

translations and rotations,

$$T_x, T_y, T_z, R_x, R_y, R_z. \quad (2.1)$$

This representation gives a transparent description of the FR3's 7-DOF anthropomorphic structure and facilitates the integration of link-side torque sensing into the kinematic chain.

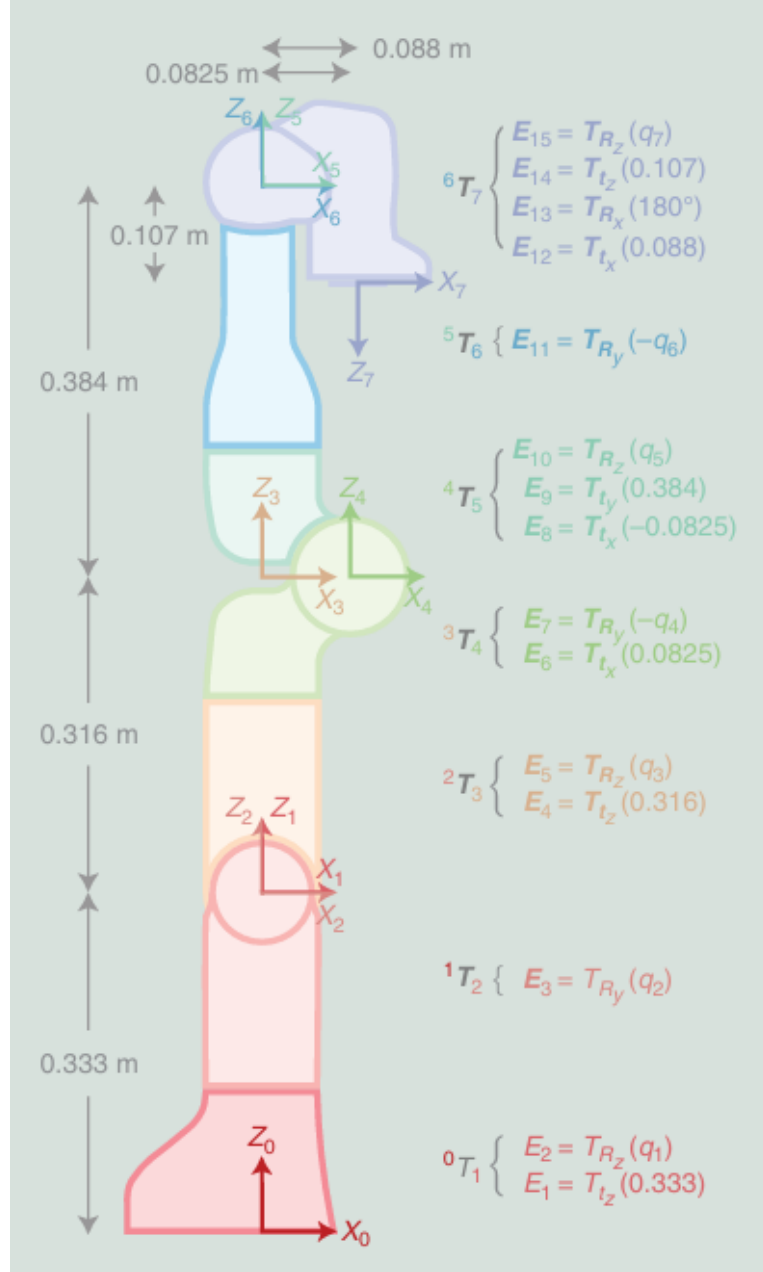


Figure 2.1: Elementary Transform Sequence of a 7-DOF Franka Emika Panda/FR3-type serial-link manipulator in its zero-angle configuration.

2.2 Forward and Inverse Kinematics

Kinematics describes the motion of a robotic manipulator without directly considering the forces and torques that generate this motion. For the Franka Research 3, which is a serial robotic manipulator with seven revolute joints, kinematics is essential because it provides the mathematical relationship between the joint variables and the pose of the end-effector. This relationship is used in trajectory generation, motion planning, Cartesian control, impedance control, admittance control, and hybrid force/position control.

The joint configuration of the FR3 is represented by the vector:

$$q = [q_1 \ q_2 \ q_3 \ q_4 \ q_5 \ q_6 \ q_7]^T \in \mathbb{R}^7$$

where q_i is the position of joint i . Since all seven joints of the FR3 are revolute, each joint variable represents an angular displacement.

Forward kinematics maps the joint-space coordinates $q \in \mathbb{R}^7$ to the pose of the end-effector in operational space. The end-effector pose is usually represented as an element of the Special Euclidean group $SE(3)$, which contains both position and orientation:

$${}^0T_e(q) = \begin{bmatrix} {}^0R_e(q) & {}^0p_e(q) \\ 0 & 1 \end{bmatrix} \in SE(3)$$

where ${}^0R_e(q)$ is the rotation matrix of the end-effector with respect to the base frame, and ${}^0p_e(q)$ is the Cartesian position of the end-effector.

Using the Elementary Transform Sequence (ETS) framework, the pose of the end-effector is computed as the product of elementary homogeneous transformation matrices from the base frame to the flange frame:

$${}^0T_e(q) = T_1(q_1)T_2(q_2) \cdots T_7(q_7)T_e$$

Each transformation $T_i(q_i)$ describes the rotation or translation associated with a link or joint of the robot. The final transformation T_e represents the fixed transformation from the last robot link to the end-effector or tool frame. This formulation is especially useful because it gives a clear and systematic description of the serial kinematic chain of the FR3.

Forward kinematics is used when the joint angles are known and the end-effector pose must be computed. For example, during an experiment, the robot continuously measures its joint positions through its encoders. By applying the forward kinematics model, the controller can compute the current Cartesian pose of the end-effector. This is required in Cartesian motion control, contact experiments, impedance control, and

force/position hybrid control.

Inverse kinematics solves the opposite problem. Given a desired end-effector pose, inverse kinematics determines the joint configuration that allows the robot to reach that pose. For a desired Cartesian pose $T_d \in SE(3)$, the inverse kinematics problem can be written as:

$${}^0T_e(q) = T_d$$

or equivalently:

$$f(q) = x_d$$

where $f(q)$ represents the forward kinematics function and x_d is the desired task-space pose.

For the FR3, inverse kinematics is more complex than for a non-redundant 6-DOF manipulator because the FR3 has seven degrees of freedom. A general pose in 3D space requires six variables: three for position and three for orientation. Since the FR3 has one additional degree of freedom, the inverse-kinematics problem generally has infinitely many solutions. This redundancy allows the robot to reach the same end-effector pose with different elbow configurations.

In practice, inverse kinematics is often solved numerically. A common approach is to linearize the relationship between joint motion and Cartesian motion using the Jacobian matrix:

$$\Delta x \approx J(q)\Delta q$$

where Δx is the Cartesian pose error and Δq is the required joint correction. The joint update can then be computed iteratively until the Cartesian error becomes sufficiently small.

For redundant robots, a common solution is based on the pseudoinverse of the Jacobian:

$$\Delta q = J^\dagger(q)\Delta x$$

where $J^\dagger(q)$ is the Moore–Penrose pseudoinverse. However, near singular configurations, the pseudoinverse may produce very large joint updates. For this reason, this thesis considers numerical methods such as damped least squares (DLS), which improves stability near singularities.

The DLS joint velocity solution is:

$$\dot{q} = J^T \left(J J^T + \lambda^2 I \right)^{-1} v$$

where λ is the damping coefficient and v is the desired Cartesian velocity or pose correction. The damping term $\lambda^2 I$ prevents numerical instability when the Jacobian becomes close to singular.

The damping coefficient controls the trade-off between accuracy and stability. A small value of λ makes the solution behave similarly to the standard pseudoinverse, giving accurate tracking when the robot is far from singularities. A larger value of λ reduces excessive joint velocities near singularities, but it may also reduce Cartesian tracking accuracy.

In an iterative inverse-kinematics algorithm, the joint configuration is updated as:

$$q_{k+1} = q_k + \Delta q_k$$

where:

$$\Delta q_k = J^T(q_k) \left(J(q_k) J^T(q_k) + \lambda^2 I \right)^{-1} \Delta x_k$$

and Δx_k is the pose error at iteration k . The iteration stops when the pose error is below a predefined threshold or when the maximum number of iterations has been reached.

For the FR3, inverse kinematics is not only used to reach a desired position. It is also useful for choosing a safe and efficient posture. Because the robot is redundant, the controller can select among multiple possible joint configurations by considering additional criteria, such as avoiding joint limits, avoiding obstacles, improving manipulability, reducing joint motion, or keeping the elbow in a preferred region of the workspace.

Therefore, forward and inverse kinematics provide the foundation for all higher-level control strategies used in this thesis. Forward kinematics allows the robot to estimate where its end-effector is, while inverse kinematics allows the controller to compute how the joints should move in order to achieve a desired Cartesian task.

2.3 The Jacobian Matrix

The Jacobian matrix is the fundamental operator of differential kinematics. While forward kinematics relates joint positions to end-effector pose, the Jacobian relates joint velocities to end-effector velocity. For the FR3, the Jacobian is especially important because it is used not only for motion control, but also for force control and torque mapping.

The differential kinematic relation is:

$$v = J(q)\dot{q}$$

where v is the Cartesian velocity, $J(q)$ is the geometric Jacobian, and $\dot{q}$ is the vector of joint velocities. For a seven-degree-of-freedom manipulator such as the FR3, the joint velocity vector is:

$$\dot{q} = [\dot{q}_1 \quad \dot{q}_2 \quad \dot{q}_3 \quad \dot{q}_4 \quad \dot{q}_5 \quad \dot{q}_6 \quad \dot{q}_7]^T \in \mathbb{R}^7$$

The Cartesian velocity of the end-effector is usually expressed as a six-dimensional twist:

$$v = \begin{bmatrix} v_x \\ v_y \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \in \mathbb{R}^6$$

where v_x, v_y, v_z are the linear velocity components and $\omega_x, \omega_y, \omega_z$ are the angular velocity components. Therefore, for the FR3, the Jacobian has dimension:

$$J(q) \in \mathbb{R}^{6 \times 7}$$

The Jacobian can be separated into two parts:

$$J(q) = \begin{bmatrix} J_v(q) \\ J_\omega(q) \end{bmatrix}$$

where $J_v(q)$ maps joint velocities to linear end-effector velocity, and $J_\omega(q)$ maps joint velocities to angular end-effector velocity.

For a revolute joint i , the contribution of the joint to the Jacobian can be written as:

$$J_{v_i} = z_{i-1} \times (p_e - p_{i-1})$$

$$J_{\omega_i} = z_{i-1}$$

where z_{i-1} is the axis of rotation of joint i , p_{i-1} is the position of joint i , and p_e is the position of the end-effector. This shows that each joint contributes to the end-effector velocity depending on its axis and its position relative to the end-effector.

The Jacobian is used in several important ways. First, it is used for velocity control. If a desired Cartesian velocity v_d is given, the corresponding joint velocity can be computed using a pseudoinverse:

$$\dot{q} = J^\dagger(q)v_d$$

Second, the Jacobian is used in inverse kinematics, where small Cartesian pose corrections are converted into joint corrections. Third, it is used in force control through the principle of force–velocity duality.

In force control, an external wrench applied at the end-effector can be mapped to joint torques using the transpose of the Jacobian:

$$\tau = J^T(q)F$$

where:

$$F = \begin{bmatrix} F_x & F_y & F_z & M_x & M_y & M_z \end{bmatrix}^T$$

is the Cartesian wrench acting on the end-effector. The vector F contains three force components and three moment components. The resulting joint torque vector is:

$$\tau = \begin{bmatrix} \tau_1 & \tau_2 & \tau_3 & \tau_4 & \tau_5 & \tau_6 & \tau_7 \end{bmatrix}^T$$

This relationship is very important for the FR3 because the robot supports torque-level control. If a desired Cartesian interaction force is specified, the controller can compute the required joint torques using the Jacobian transpose. This is the basic principle used in Cartesian impedance control and hybrid force/position control.

The Jacobian is also used to estimate the effect of external forces. If an external force acts at the end-effector, it creates torques at the joints. By comparing measured joint torques with model-predicted torques, the FR3 can estimate external interaction forces and torques. This makes the Jacobian an essential link between the robot’s internal torque sensors and the Cartesian force-control task.

Another important use of the Jacobian is the analysis of manipulability. The manipulability of a robot describes how easily the end-effector can move in different Cartesian directions. A common manipulability measure is:

$$w(q) = \sqrt{\det(J(q)J^T(q))}$$

When $w(q)$ is large, the robot can move well in many Cartesian directions. When $w(q)$ approaches zero, the robot is close to a singularity. For the FR3, manipulability is useful for selecting safe postures and avoiding configurations where motion or force control becomes difficult.

Therefore, the Jacobian matrix is central to the control framework of this thesis. It connects joint-space motion to Cartesian motion, Cartesian forces to joint torques, and redundancy resolution to task performance. Without the Jacobian, it

would not be possible to formulate Cartesian impedance control, admittance control, hybrid force/position control, or null-space posture optimization.

2.4 Redundancy and Null-Space Projection

A robotic manipulator is kinematically redundant when it has more degrees of freedom than the number of variables required to describe a given task. In terms of the spaces defined above, a manipulator is intrinsically redundant when the dimension of the task space is smaller than the dimension of the joint space, i.e., $m < n$. Redundancy is task-dependent: the same manipulator may be redundant for one task and non-redundant for another.

The Franka Research 3 is a redundant manipulator because it has seven degrees of freedom, while a general end-effector pose in three-dimensional space requires six variables: three for position and three for orientation. Therefore, for a full Cartesian pose task:

$$n = 7, \quad m = 6$$

and:

$$n > m$$

This means that the FR3 has one extra degree of freedom. This extra degree of freedom does not change the end-effector pose directly, but it allows the robot to modify its internal posture while keeping the end-effector at the same position and orientation.

Redundancy makes a robotic manipulator more dexterous and adaptable. The human arm is a familiar example, with approximately seven main degrees of freedom: three at the shoulder, one at the elbow, and three at the wrist. If the position and orientation of the hand are fixed, only six degrees of freedom are required, so the elbow can still move because of the additional degree of freedom. This makes obstacle avoidance and comfortable posture selection possible.

Similarly, the FR3 can use its redundant degree of freedom to move the elbow or change its posture while keeping the end-effector fixed. This is useful in several situations:

- avoiding obstacles in the workspace,
- avoiding joint limits,
- avoiding self-collisions,
- improving manipulability,

- keeping a comfortable elbow posture,
- reducing unnecessary joint motion,
- increasing safety during human–robot interaction.

When a manipulator is redundant, the Jacobian matrix has more columns than rows. For the FR3:

$$J(q) \in \mathbb{R}^{6 \times 7}$$

As a result, the differential kinematic equation:

$$v = J(q)\dot{q}$$

has infinitely many solutions for $\dot{q}$, provided that the desired Cartesian velocity v is feasible. The Moore–Penrose pseudoinverse gives the minimum-norm solution:

$$\dot{q} = J^\dagger v$$

However, because the robot is redundant, additional joint motion can be added without changing the end-effector velocity. The general solution is:

$$\dot{q} = J^\dagger v + (I - J^\dagger J) \dot{q}_0$$

The first term:

$$J^\dagger v$$

performs the primary Cartesian task. The second term:

$$(I - J^\dagger J) \dot{q}_0$$

lies in the null space of the Jacobian and therefore does not affect the end-effector velocity.

The matrix:

$$N = I - J^\dagger J$$

is called the null-space projection matrix. It projects any arbitrary joint velocity vector $\dot{q}_0$ into the null space of the Jacobian. This means that:

$$J(q)N = 0$$

and therefore:

$$J(q)N\dot{q}_0 = 0$$

This proves that the null-space motion does not contribute to the end-effector velocity. It only changes the internal configuration of the robot.

The vector $\dot{q}_0$ can be chosen according to a secondary objective. For example, to avoid joint limits, a potential function can be defined that becomes large when a joint approaches its limit. The robot can then move in the direction that reduces this potential while still performing the main Cartesian task.

A common form is:

$$\dot{q}_0 = -k_0 \nabla H(q)$$

where $H(q)$ is a secondary cost function and k_0 is a positive gain. The complete redundancy-resolution law becomes:

$$\dot{q} = J^\dagger v - (I - J^\dagger J) k_0 \nabla H(q)$$

If $H(q)$ represents joint-limit avoidance, the null-space term moves the joints away from their limits. If $H(q)$ represents obstacle avoidance, the null-space term moves the robot body away from obstacles. If $H(q)$ represents posture optimization, the null-space term drives the robot toward a preferred configuration.

For the purposes of this thesis, redundancy is especially important because force-control tasks usually define a primary task at the end-effector. For example, during surface contact, the robot may need to maintain a desired force at the tool while moving along a surface. At the same time, the robot can use its null space to keep a safer posture, avoid joint limits, or improve manipulability. This is one of the main advantages of using a 7-DOF manipulator such as the FR3 instead of a non-redundant 6-DOF industrial robot.

In physical human–robot interaction, redundancy also improves safety. If a human approaches the robot, the FR3 can modify its elbow configuration without disturbing the end-effector task. This allows the robot to react more naturally in shared workspaces. Therefore, null-space projection is not only a mathematical tool, but also a practical mechanism for safer and more adaptive robotic behavior.

2.5 Singularities

Singularities are configurations of a robotic manipulator where the robot loses the ability to move or apply force in one or more Cartesian directions. Mathematically, a singularity occurs when the Jacobian matrix loses rank. For the FR3, the Jacobian

has dimension:

$$J(q) \in \mathbb{R}^{6 \times 7}$$

In a regular configuration, the Jacobian has full row rank:

$$\text{rank}(J) = 6$$

This means that the robot can generate arbitrary instantaneous Cartesian motion of the end-effector in all six task-space directions. A singularity occurs when:

$$\text{rank}(J) < 6$$

In that case, the robot loses at least one independent Cartesian motion direction.

Singularities are important because they affect both motion control and force control. In motion control, a desired Cartesian velocity may require extremely large joint velocities near a singularity. This can be seen from the pseudoinverse solution:

$$\dot{q} = J^\dagger v$$

When the Jacobian is close to singular, the pseudoinverse may contain very large values. As a result, even a small desired Cartesian velocity v can produce large joint velocities. This is unsafe and may violate joint velocity limits.

In force control, singularities also cause problems. The relation between Cartesian wrench and joint torque is:

$$\tau = J^T(q)F$$

Near a singularity, the robot may lose the ability to generate or regulate forces in certain Cartesian directions. This can reduce the quality of impedance control, admittance control, and hybrid force/position control. Therefore, singularities must be considered not only in motion planning, but also in interaction control.

A useful way to analyze singularities is through the singular value decomposition of the Jacobian:

$$J = U\Sigma V^T$$

where Σ contains the singular values:

$$\sigma_1, \sigma_2, \dots, \sigma_6$$

The robot is close to a singularity when the smallest singular value approaches zero:

$$\sigma_{\min} \rightarrow 0$$

A common measure of manipulability is:

$$w(q) = \sqrt{\det(JJ^T)}$$

This can also be expressed as the product of the singular values:

$$w(q) = \sigma_1 \sigma_2 \cdots \sigma_6$$

When $w(q)$ is close to zero, the robot is close to a singular configuration. Another useful measure is the condition number:

$$\kappa(J) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

A large condition number means that the Jacobian is ill-conditioned and the robot is close to a singularity.

For a 7-DOF manipulator such as the FR3, singularities can occur in several situations. One common case is when the arm is close to being fully stretched. In such a configuration, the robot may lose the ability to move the end-effector in certain directions without producing very large joint velocities. Another possible case is wrist alignment, where several joint axes become aligned and the robot loses an independent orientation direction. Singular behavior can also appear near joint limits, workspace boundaries, or configurations where the elbow and wrist geometry reduce the available motion directions.

Although the FR3 is redundant, redundancy does not eliminate singularities completely. The additional degree of freedom helps the robot avoid singular configurations by changing its internal posture, but the robot can still become singular if it reaches an unfavorable configuration. Therefore, redundancy should be used actively through null-space control or motion planning to stay away from singular regions.

Several methods can be used to handle singularities in practice. The first method is to use damped least squares instead of the pure pseudoinverse:

$$\dot{q} = J^T (JJ^T + \lambda^2 I)^{-1} v$$

The damping term $\lambda^2 I$ prevents the solution from becoming excessively large near singularities. This improves numerical stability and helps keep joint velocities within safe limits.

The second method is manipulability maximization. A secondary objective can be added in the null space in order to move the robot away from singular configurations:

$$\dot{q} = J^\dagger v + (I - J^\dagger J) \dot{q}_0$$

where:

$$\dot{q}_0 = k_m \nabla w(q)$$

and $w(q)$ is the manipulability measure. This causes the robot to move in a direction that increases manipulability, without disturbing the primary end-effector task.

The third method is to limit Cartesian velocities and joint velocities. Even if the controller computes a feasible command, the command should be saturated to avoid unsafe motion. This is particularly important in real-time experiments with the FR3, because large velocity or torque commands may trigger safety stops.

The fourth method is to plan trajectories that avoid problematic postures. In MoveIt or other motion-planning frameworks, the robot can be guided through configurations that avoid joint limits, self-collisions, and singular regions. In direct **libfranka** control, the same idea can be implemented by checking manipulability or singular values during motion.

In the context of force control, singularity avoidance is especially important. During impedance control, the robot must map Cartesian force commands to joint torques. During admittance control, the robot must convert measured forces into motion commands. During hybrid force/position control, the robot must regulate force in some directions and position in others. If the robot is near a singularity, these mappings become less reliable and may lead to unstable or inaccurate behavior.

For this reason, the experiments in this thesis should be performed in configurations that keep the FR3 away from singularities. The arm should not be fully stretched, the wrist should not be placed in an extreme alignment, and the joints should remain away from their mechanical limits. The redundancy of the FR3 can then be used to keep the robot in a safe and well-conditioned posture while the end-effector performs the primary task.

In summary, singularities represent configurations where the robot loses motion or force capability in one or more Cartesian directions. For the FR3, singularity analysis is essential because the robot is used for force-control and human-interaction tasks. By monitoring the Jacobian, using damped least squares, maximizing manipulability, applying null-space posture control, and enforcing safe velocity and torque limits, the controller can reduce the risk of singular behavior and improve the stability of real robot experiments.

2.6 Joint Space and Operational Space

Joint space describes the state of the manipulator through its joint variables,

$$q \in \mathbb{R}^n, \quad (2.2)$$

subject to joint limits and dynamic constraints such as velocity, acceleration, and torque limits. Operational space, also called task space, describes the desired position and orientation of the end-effector in $SE(3)$. The two spaces are connected by forward kinematics and differential kinematics:

$$x = f(q), \quad \dot{x} = J(q)\dot{q}. \quad (2.3)$$

Joint-space control is natural for low-level controllers because motors and sensors are located at the joints. For the FR3, a 7-DOF serial manipulator, $q \in \mathbb{R}^7$. Joint-space descriptions are especially suitable for torque control, trajectory tracking, and dynamic algorithms such as the Recursive Newton-Euler Algorithm (RNEA) and computed-torque control. Joint limits, velocities, and accelerations are critical because they determine not only where the robot can move but also how it can move safely, without excessive jerk or actuator overload.

The workspace is the set of end-effector poses that can be reached while respecting the robot's kinematic structure, joint limits, and collision constraints. In a 7-DOF manipulator such as the FR3, redundancy enables multiple inverse-kinematics solutions for the same end-effector pose. This improves dexterity and supports criteria such as avoiding joint limits, avoiding obstacles, avoiding singularities, avoiding self-collision, and maintaining a comfortable elbow posture.

For the FR3, inverse-kinematics solutions are usually selected through optimization criteria such as:

- minimizing joint displacement,
- maximizing manipulability, for example through the Yoshikawa index,
- avoiding joint limits,
- avoiding obstacles and self-collisions.

2.7 Dynamic Analysis

The dynamic model of a robotic manipulator is important for motion simulation, mechanical analysis, and the design of control algorithms. The computation of forces and torques required for typical motions provides useful information for the design of joints,

transmission systems, and actuators. In this thesis, the equations of motion of the FR3 are presented using the Newton-Euler approach, which is computationally efficient and expresses the model recursively along the open kinematic chain of the robot.

Deriving a complete dynamic model by hand for a 7-DOF manipulator such as the FR3 would be time-consuming and prone to errors. Therefore, in practical implementations, symbolic software and robot-dynamics libraries are commonly used. Dynamic analysis describes the mathematical relationship between motion variables ($q, \dot{q}, \ddot{q}$) and the joint torques τ required to execute the motion, taking into account masses, inertia tensors, gravity, friction, and external contacts.

The main symbols used in this section are:

- $q \in \mathbb{R}^n$: vector of joint variables; for the FR3, $n = 7$.
- $\dot{q}$ and $\ddot{q}$: joint velocities and joint accelerations.
- $\tau \in \mathbb{R}^n$: joint torques or generalized actuation forces.
- g : gravitational acceleration.
- T : kinetic energy; U : potential energy.
- $M(q)$: inertia matrix.
- $C(q, \dot{q})\dot{q}$: centrifugal and Coriolis terms.
- $G(q)$: gravity vector.
- $\tau_f(\dot{q})$: friction terms.
- $J(q)$: Jacobian matrix, also useful for mapping external forces.

2.7.1 Equations of Motion

The equations of motion of a robotic manipulator in joint space can be obtained either through the Lagrange method or through the Recursive Newton-Euler Algorithm. The Lagrange method is systematic and independent of the chosen coordinate frame, while RNEA produces the model in a recursive form that is computationally efficient.

RNEA is an efficient method for computing inverse dynamics in serial robotic manipulators. Instead of deriving large nonlinear expressions, it computes the required joint torques directly from $(q, \dot{q}, \ddot{q})$ using two recursive passes: a forward pass for the angular and linear velocities and accelerations of the links, and a backward pass for transferring forces and moments toward the base. Its complexity is linear in the number of degrees of freedom, $O(n)$, which makes it suitable for real-time applications such as feedforward torque control, gravity compensation, and computed-torque control on the FR3.

In general form, the dynamics can be written as

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) + \tau_f + J^T F_{\text{ext}}. \quad (2.4)$$

RNEA computes τ recursively along the robot chain. It is based on the Newton-Euler laws:

$$F = ma, \quad (2.5)$$

$$N = I\alpha + \omega \times (I\omega), \quad (2.6)$$

where F is force, N is moment, m is mass, a is linear acceleration, I is the inertia tensor, α is angular acceleration, and ω is angular velocity.

For each link, the algorithm computes angular velocities and accelerations, the linear acceleration of the center of mass, the force F_i , and the moment N_i required by the link. The calculations are performed in two stages:

1. **Forward recursion:** from the base to the end-effector, computing kinematic quantities such as ω , α , and a .
2. **Backward recursion:** from the end-effector back to the base, computing link forces, link moments, and finally the joint torques τ .

For a revolute joint i with axis z_i , the transformations from link $i-1$ to link i can be described by a rotation matrix R_i and a displacement vector p_i . The base conditions are usually set as

$$\omega_0 = 0, \quad \alpha_0 = 0, \quad a_0 = -g. \quad (2.7)$$

For every link i , a typical forward recursion can be expressed as

$$\omega_i = R_i^T \omega_{i-1} + z_i \dot{q}_i, \quad (2.8)$$

$$\alpha_i = R_i^T \alpha_{i-1} + z_i \ddot{q}_i + \omega_i \times (z_i \dot{q}_i), \quad (2.9)$$

$$a_i = R_i^T (a_{i-1} + \alpha_{i-1} \times p_i + \omega_{i-1} \times (\omega_{i-1} \times p_i)), \quad (2.10)$$

$$a_{c,i} = a_i + \alpha_i \times r_{c,i} + \omega_i \times (\omega_i \times r_{c,i}), \quad (2.11)$$

where $r_{c,i}$ is the vector from the origin of frame i to the center of mass of link i .

The internal force and moment of link i are

$$F_i = m_i a_{c,i}, \quad (2.12)$$

$$N_i = I_{c,i} \alpha_i + \omega_i \times (I_{c,i} \omega_i). \quad (2.13)$$

The backward recursion combines the forces and moments from the subsequent links:

$$f_i = R_{i+1}f_{i+1} + F_i, \quad (2.14)$$

$$n_i = N_i + R_{i+1}n_{i+1} + r_{c,i} \times F_i + p_{i+1} \times (R_{i+1}f_{i+1}). \quad (2.15)$$

For a revolute joint, the joint torque is obtained by projecting the moment onto the joint axis:

$$\tau_i = n_i^T z_i. \quad (2.16)$$

For a prismatic joint, the generalized force would be

$$\tau_i = f_i^T z_i. \quad (2.17)$$

2.7.2 Forward Dynamics and Inverse Dynamics

The Newton-Euler method provides the relationship between joint torques, possible end-effector forces, and the motion of the robot. The forward-dynamics problem consists of determining the joint accelerations $\ddot{q}(t)$ for $t > t_0$ caused by given joint torques $\tau(t)$ and possible end-effector forces $h_e(t)$, when the initial positions $q(t_0)$ and velocities $\dot{q}(t_0)$ are known. The joint velocities and positions are obtained by integrating the nonlinear differential equations of motion.

The inverse-dynamics problem consists of determining the joint torques $\tau(t)$ required to produce a desired motion, defined by q , $\dot{q}$, and $\ddot{q}$, while also considering any external forces acting on the end-effector. Inverse dynamics is useful for trajectory planning and control because, once the desired joint trajectory is known, it provides the torques that must be applied to the joints in order to generate the desired motion.

For a robot with n joints, forward dynamics typically requires $O(n^2)$ operations, while inverse dynamics using RNEA requires $O(n)$ operations. This computational efficiency is one of the reasons RNEA is commonly used in real-time robotic control.

To achieve smooth interaction, the controller must account for the robot dynamics. A commonly used form of the Euler-Lagrange model is

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau_{\text{cmd}} - \tau_{\text{ext}}, \quad (2.18)$$

where $M(q)$ is the inertia matrix, $C(q, \dot{q})\dot{q}$ contains Coriolis and centrifugal effects, and $g(q)$ is the gravity vector. The high-quality dynamic model of the FR3 enables gravity compensation, allowing the arm to float and providing the dynamic transparency required for sensitive human-robot interaction.

2.8 Force Sensors

The measurement of a force or torque is commonly reduced to the measurement of the deformation caused by that force or torque on an elastic element with known properties. Therefore, an indirect force measurement can be achieved by measuring very small displacements or strains. The basic component of many force sensors is the strain gauge, which uses the change in electrical resistance of a conductor when it is deformed.

Force sensing is a fundamental requirement in robotic systems that interact physically with their environment. In classical industrial manipulators, the robot often operates mainly through position control, and contact with the environment is either avoided or treated as an error. In collaborative and contact-rich robotics, however, contact forces become part of the task itself. For example, when a robot presses on a surface, grasps an object, slides an object on a table, or reacts to a human push, the controller must be able to estimate and regulate the physical interaction between the robot and the environment.

In general, a force sensor converts mechanical deformation into an electrical signal. A common sensing principle is based on strain gauges, where a small deformation of an elastic element changes its electrical resistance. From this deformation, the applied force or torque can be estimated. Traditional robotic force sensing is often implemented using a six-axis force/torque sensor mounted between the robot flange and the end-effector. Such a sensor measures three force components and three torque components at the wrist of the robot.

In the Franka Research 3, force sensing is implemented in a more integrated way. The FR3 is a seven-degree-of-freedom robotic manipulator equipped with link-side torque sensors in all seven joints. This means that each joint can directly measure the torque transmitted through the mechanical structure of the arm. As a result, the robot can detect external interactions not only at the end-effector, but also along the body of the robot. This is important for human-robot interaction, because a human may touch or push the robot at any link, not only at the tool.

The use of joint torque sensors gives the FR3 a form of proprioceptive force sensing. Proprioception means that the robot senses interaction through its own internal joints, rather than only through an external force sensor. The measured joint torque vector can be written as:

$$\tau_J = [\tau_1 \quad \tau_2 \quad \tau_3 \quad \tau_4 \quad \tau_5 \quad \tau_6 \quad \tau_7]^T$$

where τ_i is the measured torque at joint i . In the Franka Control Interface and `libfranka`, this quantity is available through the robot state as:

`state.tau_J`

which represents the measured link-side joint torque sensor signals in Nm. The external torque estimate is also available as:

`state.tau_ext_hat_filtered`

This value represents a filtered estimate of the external torques acting on the robot joints. Conceptually, the external torque estimate is obtained by comparing the measured joint torques with the torques expected from the robot dynamic model. If the robot model predicts the torque needed for gravity, inertia, Coriolis effects, and commanded motion, then any significant difference between the expected torque and the measured torque can indicate an external interaction. This can be expressed as:

$$\hat{\tau}_{ext} = \tau_J - \tau_{model}$$

where:

$$\tau_{model} = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q)$$

Here, $M(q)$ is the inertia matrix, $C(q, \dot{q})\dot{q}$ represents Coriolis and centrifugal effects, and $g(q)$ represents gravity. Therefore, the external torque estimate is not a completely independent measurement like a standalone sensor. It is a model-based estimate that depends on the accuracy of the robot model, payload configuration, friction compensation, and sensor calibration.

For the purposes of this thesis, this sensing architecture is highly useful because it allows force-control experiments without adding a separate wrist force/torque sensor. The FR3 can estimate external forces and torques through its internal sensors and dynamic model, which reduces hardware complexity and keeps the end-effector lighter. This is especially useful in tasks such as hand-guiding, compliant motion, surface contact, obstacle detection, and human-robot interaction.

However, the internal torque-sensing approach also has limitations. The robot can detect that an external interaction exists, but the exact point of contact on the robot body is not always directly known. In addition, the force estimate may be affected by modeling errors, unmodeled friction, cable forces, payload errors, or small offsets in the estimated bias. For this reason, practical force-control experiments usually require bias compensation, filtering, conservative gains, and safety limits.

In this thesis, the FR3 torque and force estimates were used for reading external interaction forces, detecting contact with the environment, implementing compliant behavior, applying normal force during surface manipulation, monitoring external joint

torques, and evaluating the safety of human-robot interaction. Therefore, the FR3's integrated torque sensing is not only a measurement feature, but also a central part of the control architecture used in this work.

2.9 Wrench Force Sensors

When the end-effector of a robot is in contact with the environment, a wrench force sensor can measure the three components of force and the three components of torque with respect to a coordinate frame attached to the sensor or end-effector. This information is useful for contact detection, force regulation, assembly tasks, polishing, grinding, and other operations that require controlled interaction with the environment.

A wrench is a six-dimensional vector that combines the three components of force and the three components of torque acting on a rigid body. In robotics, the wrench applied at the end-effector is usually written as:

$$W = \begin{bmatrix} F_x \\ F_y \\ F_z \\ M_x \\ M_y \\ M_z \end{bmatrix}$$

where F_x , F_y , and F_z are the Cartesian force components, measured in Newtons, and M_x , M_y , and M_z are the moment components, measured in Newton-meters. A classical six-axis force/torque sensor mounted at the wrist of a robot directly measures this wrench at the interface between the robot and the end-effector.

In the FR3, the external wrench can be estimated without necessarily mounting an additional wrist force/torque sensor. Through libfranka, the robot state provides the estimated external wrench acting on the stiffness frame K . Two important variables are:

`state.O_F_ext_hat_K`

and:

`state.K_F_ext_hat_K`

The first one, `state.O_F_ext_hat_K`, is the estimated external wrench acting on the stiffness frame and expressed relative to the robot base frame O . The second one, `state.K_F_ext_hat_K`, is the same estimated external wrench expressed relative to the stiffness frame K . In both cases, the vector contains six values with units:

$$[N, N, N, Nm, Nm, Nm]$$

This distinction between frames is important. When the wrench is expressed in the base frame O , the force directions are interpreted according to the fixed robot base coordinate system. When the wrench is expressed in the stiffness frame K , the force directions are interpreted according to the local frame used for Cartesian impedance behavior. For example, if the robot performs a surface-contact task, the vertical force component may be easier to interpret in the base frame if the table normal is aligned with the base z -axis.

The relation between a Cartesian wrench and joint torques is given by the Jacobian transpose:

$$\tau_{ext} = J(q)^T W$$

where $J(q)$ is the geometric Jacobian of the robot. This equation shows how a force or torque applied at the end-effector produces torques at the joints. Conversely, if external joint torques are estimated, the robot can use its kinematic and dynamic model to estimate the corresponding Cartesian wrench.

For force-control applications, the wrench estimate is extremely useful because it allows the controller to regulate contact forces in task space. For example, in a surface manipulation task, the controller may regulate only the vertical force component F_z , while still controlling the horizontal position of the end-effector. This leads to a hybrid force/position control structure:

$$W = \begin{bmatrix} F_x^{pos} \\ F_y^{pos} \\ F_z^{force} \\ M_x^{ori} \\ M_y^{ori} \\ M_z^{ori} \end{bmatrix}$$

In this structure, F_x and F_y can be generated by Cartesian position control, F_z can be generated by force feedback, and the moment components can be used to maintain the desired end-effector orientation.

For the FR3 experiments in this thesis, the most important wrench component was the vertical force F_z , because the robot had to maintain contact with a table while sliding an object. The measured raw force was read from:

$$F_{z,raw} = \text{state.0_F_ext_hat_K}[2]$$

However, raw force readings may contain offsets due to gravity compensation, pay-

load configuration, sensor drift, or the initial contact condition. For this reason, bias compensation was used before force control:

$$F_{z,bias} = \frac{1}{N} \sum_{i=1}^N F_{z,raw,i}$$

Then the compensated force was computed as:

$$F_{z,\Delta} = F_{z,raw} - F_{z,bias}$$

Depending on the sign convention and the direction of contact, the measured contact force was written as:

$$F_{z,contact} = s_c F_{z,\Delta}$$

where s_c is a sign coefficient selected experimentally. In the implemented experiments, downward contact with the table produced a negative change in the raw F_z signal, so the sign coefficient was chosen as:

$$s_c = -1$$

This allowed the controller to interpret a downward pressing contact as a positive normal contact force.

The desired normal force was then compared with the measured contact force:

$$e_F = F_{z,target} - F_{z,contact}$$

and the commanded vertical force was computed using a feedback term:

$$F_{z,cmd} = K_f e_F - D_f \dot{z}$$

where K_f is the force-control gain and D_f is the damping term used to reduce oscillations during contact. Finally, the Cartesian wrench command was converted into joint torques using:

$$\tau_{cmd} = J(q)^T W_{cmd} + C(q, \dot{q})$$

where $C(q, \dot{q})$ represents Coriolis compensation.

In conclusion, although the FR3 does not require an external six-axis wrist force/torque sensor for the experiments of this thesis, it can still provide wrench information through its internal joint torque sensors and model-based estimation. This makes the robot especially suitable for force control, impedance control, hand-guiding, collision detection, and safe physical human-robot interaction. The wrench estimate is therefore a key mea-

surement for transforming the FR3 from a purely motion-controlled manipulator into a force-aware collaborative robotic system.

Chapter 3

Force Control and Sensing

A basic requirement for the success of a robotic task is the ability to handle the interaction between the manipulator and its environment. The most effective quantity for describing the state of this interaction is the contact force at the end-effector. Large contact forces are generally undesirable because they can overstress the robot, damage the object, or create unsafe conditions for nearby humans.

3.1 Manipulator Interaction with the Environment

The control of interaction between a robotic manipulator and its environment is critical for many practical tasks in which the end-effector must manipulate an object or operate on a surface, such as polishing, cutting, machining, or assembly. During interaction, the environment imposes constraints on the geometric paths that the end-effector can follow. This situation is called constrained motion, and a pure position-control strategy may fail because small position errors can create large contact forces.

Interaction-control strategies are commonly classified into two categories: indirect force control and direct force control. In indirect force control, the force is regulated through motion control without an explicit force-feedback loop. In direct force control, the controller includes force feedback and can regulate the contact force toward a desired value. Compliance control and impedance control belong to the first category and are studied in detail in this thesis.

The contact force is the quantity that best describes the interaction state. The availability of force measurements improves the performance of interaction control. Planning errors can increase the contact force and cause the end-effector to deviate from the desired trajectory. The controller may then react by attempting to reduce the position error, which can further increase the contact force until actuator saturation or damage occurs. This problem becomes more likely when the environment is stiff and the position controller is very accurate. The drawback can be reduced by ensuring compliant behavior during contact.

3.2 Force Control

This thesis focuses mainly on methods that implement indirect force control, because the interaction force is controlled through the desired end-effector motion and impedance behavior. The interaction between the robot and the environment is affected directly by the compliance of the environment and by the compliance or impedance of the manipulator. A force-control system usually includes a control law based not only on force measurements but also on velocity and position measurements.

The effectiveness of such methods depends on the geometry of the contact and on the specific interaction case. A force-control strategy is meaningful only in those directions of the operational space in which interaction forces can actually appear between the manipulator and the environment.

3.3 Physical and Artificial Constraints

For many robotic tasks, it is possible to define a time-varying orthogonal coordinate system in which the environmental constraints can be identified clearly. This coordinate system is called the constraint frame and can be denoted by

$$O_c = x_c y_c z_c. \quad (3.1)$$

Two degrees of freedom are associated with each axis of this frame: one translational or force-related degree of freedom along the axis and one rotational or torque-related degree of freedom about the axis.

For each degree of freedom, the environment imposes either a velocity constraint, meaning that it does not allow motion along a direction or rotation about an axis, or a force constraint, meaning that it does not allow the application of arbitrary forces or torques along that degree of freedom. Such constraints are called physical constraints because they are determined directly by the geometry of the task.

The robot can control only the variables that are not subject to physical constraints. The reference values assigned to these controllable variables are called artificial constraints because they are imposed by the chosen strategy for executing the task. When friction is large, forces and torques may also appear in directions associated with physical force constraints.

3.4 Compliance Control

Compliance control is the part of robot control that prevents the manipulator from behaving as a perfectly rigid mechanism during contact. Instead of resisting all external

forces, the robot yields, absorbs energy, and interacts safely and stably with humans, objects, and surfaces. This capability is one of the key strengths of the FR3 because the robot is designed for collaborative operation and torque control.

A rigid manipulator attempts to hold its commanded position almost independently of external forces. During contact, this behavior can become dangerous or lead to large loads and instability. A compliant manipulator, on the other hand, allows its motion or position to depend on the external force, as if a virtual spring and damper existed between the tool and the target. The goal is safe contact, stable behavior, soft interaction, and improved adaptation to uncertainty.

The three main categories of compliance control are impedance control, admittance control, and hybrid force/position control. In admittance control, the controller converts an external force into a desired motion according to a virtual dynamic model:

$$M_a\ddot{x}_c + D_a\dot{x}_c + K_ax_c = F_{\text{ext}}, \quad (3.2)$$

and then defines the desired position as

$$x_d = x_{\text{ref}} + x_c. \quad (3.3)$$

Admittance control is commonly preferred when the robot is mainly position-controlled and compliance must be generated externally. In torque-controlled robots such as the FR3, impedance control is often more natural, although admittance control can also be implemented.

3.5 Impedance Control

The FR3 is fundamentally a torque-controlled manipulator, which distinguishes it from traditional position-controlled industrial robots. Through the Franka Control Interface, the robot allows direct torque commands at each joint at a frequency of 1 kHz.

In this research, an impedance-control strategy is implemented. The goal is to regulate the relationship between the end-effector pose error and the interaction wrench. The controller is modeled as a virtual mass-spring-damper system. A typical Cartesian impedance behavior is expressed as

$$M_d(\ddot{x} - \ddot{x}_d) + D_d(\dot{x} - \dot{x}_d) + K_d(x - x_d) = F_{\text{ext}}, \quad (3.4)$$

where x is the actual Cartesian pose of the tool, x_d is the desired pose, F_{ext} is the external wrench, M_d is the virtual inertia, D_d is the virtual damping, and K_d is the virtual stiffness.

A common torque command for Cartesian impedance control can be written as

$$\tau_{\text{cmd}} = J^T (K(x_d - x) + D(\dot{x}_d - \dot{x})) + g(q) + C(q, \dot{q})\dot{q}. \quad (3.5)$$

This approach ensures that the robot behaves compliantly during human contact and effectively transforms the manipulator into a programmable soft mechanism that can yield to external disturbances.

The interpretation of the controller parameters is intuitive. A larger stiffness matrix K_d makes the robot more rigid and less compliant, while a smaller K_d makes it softer and more compliant. The damping matrix D_d reduces oscillations and helps stabilize contact. This makes impedance control particularly suitable for surface-contact tasks such as polishing, grinding, and assembly.

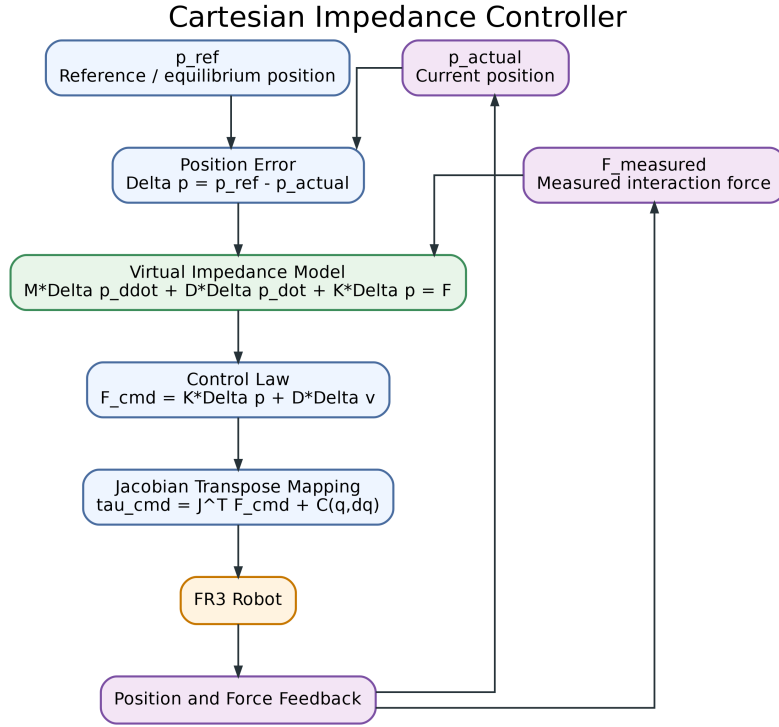


Figure 3.1: Cartesian impedance controller block diagram.

The above block diagram presents a Cartesian impedance controller. This controller does not command the robot to behave as a perfectly rigid position-controlled system. Instead, it defines a desired dynamic relationship between the Cartesian position error and the interaction force.

The main idea is to make the end-effector behave like a virtual mechanical system composed of a mass, a spring, and a damper. The general impedance model is:

$$M\Delta\ddot{p} + D\Delta\dot{p} + K\Delta p = F$$

where M is the virtual inertia matrix, D is the virtual damping matrix, K is the virtual stiffness matrix, and F is the interaction force. The position deformation is defined as:

$$\Delta p = p_{ref} - p_{actual}$$

where p_{ref} is the desired reference position and p_{actual} is the measured current position of the end-effector.

In the block diagram, the controller starts from a desired Cartesian reference position:

$$p_{ref}$$

This reference is not necessarily treated as a strict position command. Instead, it is interpreted as an equilibrium position. If no external force is applied, the robot tends to remain close to this equilibrium. If an external force is applied, the robot reacts according to the selected stiffness and damping values.

A simplified Cartesian impedance control law can be written as:

$$F_{cmd} = K(p_{ref} - p_{actual}) + D(\dot{p}_{ref} - \dot{p}_{actual})$$

If the desired velocity is zero, this becomes:

$$F_{cmd} = K(p_{ref} - p_{actual}) - D\dot{p}_{actual}$$

The generated Cartesian force command is then mapped into joint torques using the Jacobian transpose:

$$\tau_{cmd} = J^T(q)F_{cmd} + C(q, \dot{q})$$

where $C(q, \dot{q})$ represents Coriolis compensation. In practical implementations, gravity compensation and torque-rate saturation may also be included for safety and stability.

The impedance controller is very important for the FR3 because the robot supports torque-level control and has integrated torque sensors in its joints. This allows the robot to behave compliantly during contact. If the stiffness K is high, the robot behaves more rigidly and strongly resists displacement. If the stiffness K is low, the robot behaves more softly and allows larger displacement under the same external force. The damping D is used to reduce oscillations and improve contact stability.

The impedance controller is suitable for tasks where the robot must maintain stable contact with the environment while still reacting safely to external disturbances. Examples include surface following, physical human-robot interaction, polishing, pressing,

and compliant manipulation.

3.6 Admittance Control

Admittance control is complementary to impedance control. In admittance control, the measured external force is used as an input to a virtual dynamic system that generates a desired motion. The robot then follows that motion through its position or velocity controller. This approach is useful for stiff, high-precision manipulators that do not accept direct torque commands. Since the FR3 supports torque control, impedance control is the main focus of this thesis, but admittance control remains an important alternative for systems with different low-level control architectures.

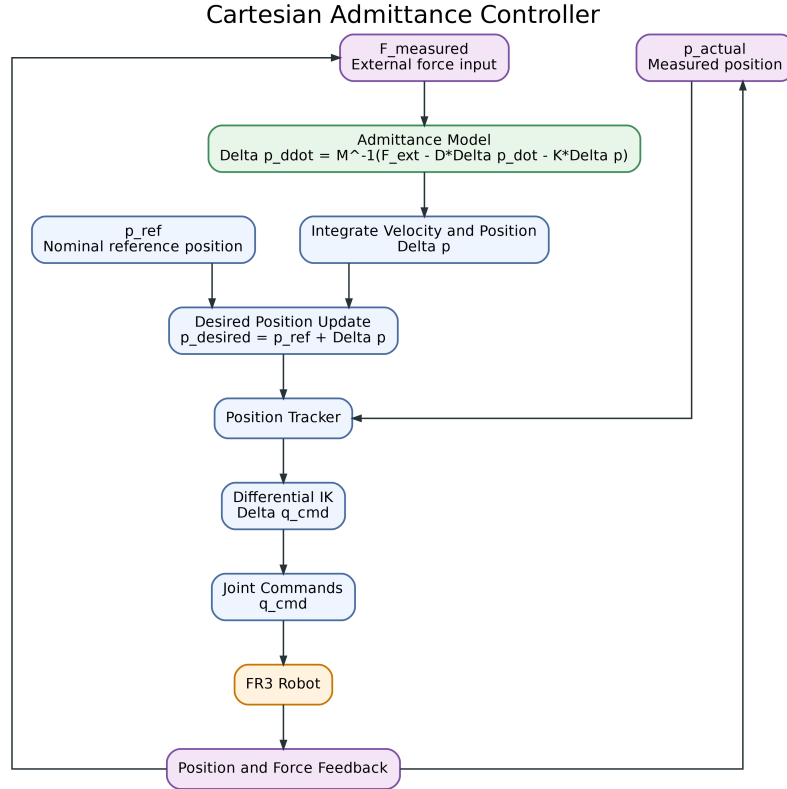


Figure 3.2: Cartesian admittance controller block diagram.

The above block diagram represents a Cartesian admittance controller. While impedance control receives a motion error and produces a force-like response, admittance control follows the opposite logic: it receives an external force measurement and produces a desired motion.

The controller uses a virtual mass-spring-damper model to convert the measured external force into a displacement command. The general admittance equation is:

$$M\Delta\ddot{p} + D\Delta\dot{p} + K\Delta p = F_{ext}$$

Solving for acceleration gives:

$$\Delta\ddot{p} = M^{-1}(F_{ext} - D\Delta\dot{p} - K\Delta p)$$

This acceleration is integrated once to obtain velocity:

$$\Delta\dot{p}(t + \Delta t) = \Delta\dot{p}(t) + \Delta\ddot{p}\Delta t$$

and integrated again to obtain position displacement:

$$\Delta p(t + \Delta t) = \Delta p(t) + \Delta\dot{p}(t + \Delta t)\Delta t$$

The final desired Cartesian position is then updated as:

$$p_{desired} = p_{ref} + \Delta p$$

This desired position is passed to a position tracker, which generates the required Cartesian or joint commands. In the diagram, the desired Cartesian motion is converted into joint commands through differential inverse kinematics:

$$\Delta q = J^\#(q)\Delta p_{desired}$$

and then integrated to produce the final joint command:

$$q_{cmd} = q_{actual} + \Delta q$$

The FR3 executes the command and returns new position and force feedback to the controller.

The main advantage of admittance control is that the robot can respond smoothly to external forces. For example, if a human pushes the end-effector, the measured external force becomes the input of the admittance model and the robot generates a corresponding compliant displacement. The magnitude of this displacement depends on the virtual parameters M , D , and K . A smaller virtual stiffness allows the robot to move more easily, while a larger virtual stiffness makes the robot resist motion more strongly.

Admittance control is particularly useful for collaborative tasks, soft contact, hand-guiding, and safe human-robot interaction. It is often used when the robot is controlled mainly through position commands but must still display compliant behavior. In the case of the FR3, admittance control can also be implemented by reading the estimated external wrench from the robot state and updating the commanded Cartesian target in real time.

3.7 Hybrid Force/Position Control

Hybrid force/position control is based on the idea that not all directions of motion should be controlled in the same way during contact. The description of an interaction task through physical and artificial constraints requires a controller that assigns desired values only to variables that are not physically constrained by the environment. The control action must not conflict with variables that are constrained by contact, otherwise contradictions between control and environment interaction may occur.

In hybrid control, the task directions are separated into position-controlled directions and force-controlled directions. For example, during a surface-contact task, the robot may apply a desired normal force, such as 10 N, while moving along the tangential directions of the surface. This separation allows the robot to maintain contact force while simultaneously following a desired path.

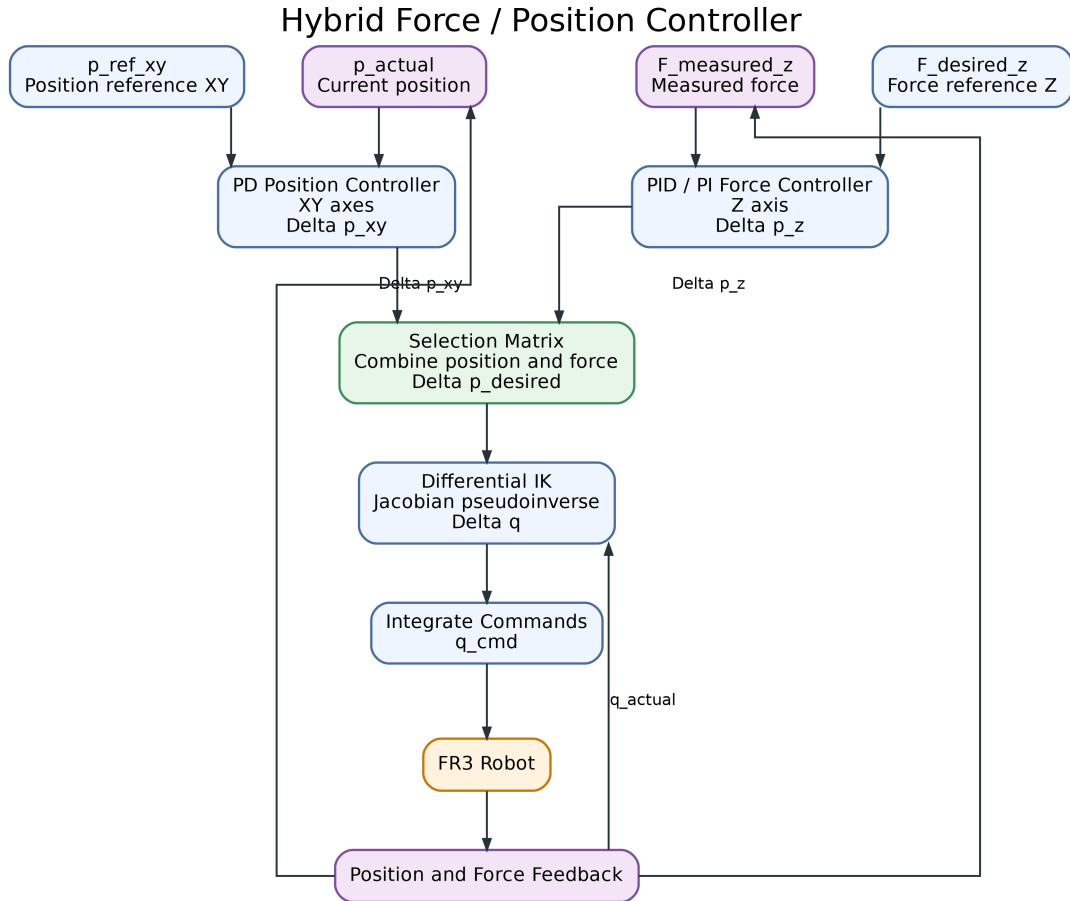


Figure 3.3: Hybrid force/position controller block diagram.

The above block diagram describes a hybrid force/position controller. This controller is designed for tasks where the robot must control motion in some directions and force in other directions at the same time. This is especially useful when the end-effector is in contact with a surface.

A typical example is a surface manipulation task. The robot may need to move along the x and y directions while maintaining a desired normal force along the z direction. In this case, the horizontal motion is controlled by position control, while the vertical contact interaction is controlled by force control.

The controller receives two different references:

$$p_{ref,xy}$$

for the desired Cartesian position in the horizontal plane, and:

$$F_{desired,z}$$

for the desired contact force along the vertical axis. In the block diagram, the desired vertical force is shown as a reference force, for example:

$$F_{desired,z} = 5N$$

The actual Cartesian position is measured as:

$$p_{actual}$$

and the measured vertical contact force is:

$$F_{measured,z}$$

The position-control part computes the Cartesian position error in the controlled position directions:

$$\Delta p_{xy} = p_{ref,xy} - p_{actual,xy}$$

A PD position controller then generates a desired correction in the x and y directions. At the same time, the force-control part compares the desired vertical force with the measured vertical force:

$$e_F = F_{desired,z} - F_{measured,z}$$

A PID or PI force controller can then be used to produce a desired correction along the z -axis. In simplified form, this force-control contribution can be written as:

$$\Delta p_z = K_F e_F + K_I \int e_F dt + K_D \dot{e}_F$$

The selection matrix is the key element of hybrid force/position control. It determines which task-space directions are position-controlled and which are force-controlled.

For example, for a task where x and y are position-controlled and z is force-controlled, the selection matrices can be conceptually written as:

$$S_p = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

and:

$$S_f = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The final desired Cartesian correction is obtained by combining the position and force contributions:

$$\Delta p_{desired} = S_p \Delta p_{pos} + S_f \Delta p_{force}$$

This desired Cartesian correction is then converted into a joint-space correction using differential inverse kinematics. The relationship between Cartesian velocity and joint velocity is:

$$\dot{x} = J(q)\dot{q}$$

Therefore, a desired Cartesian correction can be mapped to a joint correction using the pseudoinverse of the Jacobian:

$$\Delta q = J^\#(q) \Delta p_{desired}$$

where $J^\#(q)$ is the pseudoinverse of the Jacobian. The joint command is then produced by integrating the joint correction:

$$q_{cmd}(t + \Delta t) = q_{actual}(t) + \Delta q$$

The command is sent to the FR3 robot, and the robot returns updated position and force feedback. In this way, the controller continuously adjusts the robot motion so that it follows the desired path while maintaining the desired contact force.

The hybrid force/position controller is particularly suitable for assembly tasks, polishing, wiping, pushing, sliding, and surface-following operations. Its main advantage is that it explicitly separates motion control and force control according to the physical constraints of the task.

3.8 Comparison of the Controllers

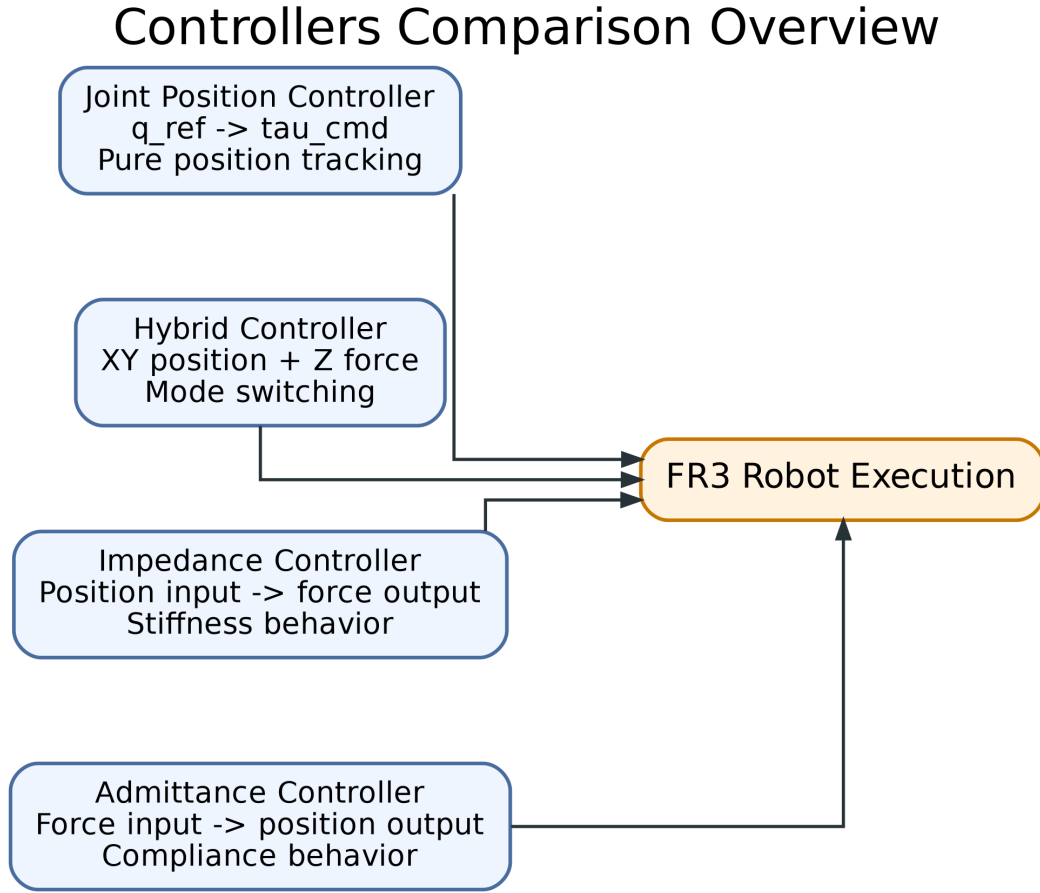


Figure 3.4: Comparison overview of the implemented FR3 controllers.

The final diagram provides an overview of the three control strategies and shows that all of them eventually lead to robot execution by the FR3. However, each controller uses a different input-output logic and is suitable for different types of tasks.

The joint space position controller uses a desired joint reference as input and produces torque or joint commands as output:

$$q_{ref} \rightarrow \tau_{cmd}$$

It is appropriate for accurate free-space motion, but it does not explicitly regulate contact forces.

The hybrid force/position controller uses both position and force references. It controls some task directions through position control and other directions through force control:

$$XY \text{ Position} + Z \text{ Force}$$

This makes it suitable for surface-contact tasks where the robot must move along a surface while applying a desired normal force.

The Cartesian impedance controller receives a desired position or equilibrium point and produces a force-like response through virtual stiffness and damping:

$$\textit{Position Input} \rightarrow \textit{Force Output}$$

This makes the robot behave like a programmable mechanical impedance. It is well suited for stable contact and interaction tasks.

The Cartesian admittance controller receives an external force as input and produces a desired position update as output:

$$\textit{Force Input} \rightarrow \textit{Position Output}$$

This makes the robot react compliantly to external forces and is especially useful for human-guided motion and soft physical interaction.

In summary, the four controllers can be interpreted as different levels of interaction capability. Joint space position control provides the basic tracking behavior. Hybrid force/position control introduces explicit force regulation in selected directions. Impedance control defines how the robot mechanically reacts to position errors and contact forces. Admittance control converts measured external forces into compliant motion. Together, these controllers provide a complete framework for studying force control and human-robot interaction with the FR3 manipulator.

3.9 Pseudo-Algorithms of the Implemented Controllers

This section presents the pseudo-algorithms of the three main control strategies implemented and studied in this thesis: Cartesian impedance control, Cartesian admittance control, and hybrid force/position control. The algorithms are written in a general form, but they follow the practical implementation logic used for the FR3 experiments. Each controller runs in a cyclic real-time loop, where the robot state is read, the control law is evaluated, safety limits are applied, and the final command is sent to the robot.

3.9.1 Pseudo-Algorithm of the Cartesian Impedance Controller

The impedance controller defines a virtual mechanical relation between the Cartesian pose error of the end-effector and the force or torque response of the robot. In this controller, the desired Cartesian pose is treated as an equilibrium pose. If the robot

is displaced from this pose because of contact or external disturbance, the controller generates a restoring wrench according to a virtual spring-damper model.

The simplified Cartesian impedance law is:

$$F_{cmd} = K_x(x_d - x) + D_x(\dot{x}_d - \dot{x}) \quad (3.6)$$

where K_x is the Cartesian stiffness matrix and D_x is the Cartesian damping matrix. The Cartesian wrench is then mapped to joint torques through:

$$\tau_{cmd} = J(q)^T F_{cmd} + C(q, \dot{q}) \quad (3.7)$$

where $J(q)$ is the Jacobian matrix and $C(q, \dot{q})$ represents Coriolis compensation.

The implemented logic of the Cartesian impedance controller can be described as follows:

Step 1: Define the desired Cartesian pose x_d and desired Cartesian velocity $\dot{x}_d$.

Step 2: Define the Cartesian stiffness matrix K_x and damping matrix D_x . These matrices determine how rigid or compliant the robot behaves during motion and contact.

Step 3: Initialize the previous torque command τ_{prev} , which is used for torque-rate saturation.

Step 4: Read the current robot state from the Franka Control Interface. The most important measured quantities are:

$$q, \quad \dot{q}, \quad x, \quad \dot{x}, \quad \tau_J, \quad \hat{\tau}_{ext} \quad (3.8)$$

Step 5: Compute the current Jacobian matrix:

$$J(q) \quad (3.9)$$

Step 6: Compute the model compensation terms, mainly the Coriolis compensation:

$$C(q, \dot{q}) \quad (3.10)$$

Step 7: Compute the Cartesian position error:

$$e_x = x_d - x \quad (3.11)$$

Step 8: Compute the Cartesian velocity error:

$$e_{\dot{x}} = \dot{x}_d - \dot{x} \quad (3.12)$$

Step 9: Compute the desired Cartesian wrench using the virtual spring-damper law:

$$F_{cmd} = K_x e_x + D_x e_{\dot{x}} \quad (3.13)$$

Step 10: Convert the Cartesian wrench into joint torques:

$$\tau_{raw} = J(q)^T F_{cmd} + C(q, \dot{q}) \quad (3.14)$$

Step 11: Apply torque-rate saturation in order to avoid sudden torque jumps:

$$\Delta\tau = \text{clip}(\tau_{raw} - \tau_{prev}, -\Delta\tau_{\max}, \Delta\tau_{\max}) \quad (3.15)$$

$$\tau_{cmd} = \tau_{prev} + \Delta\tau \quad (3.16)$$

Step 12: Send the final torque command τ_{cmd} to the FR3 through the Franka Control Interface.

Step 13: Save the relevant robot state, force, torque, and error data to the log file.

Step 14: Update the previous torque command:

$$\tau_{prev} \leftarrow \tau_{cmd} \quad (3.17)$$

Step 15: Repeat the loop until the experiment finishes or a safety stop is triggered.

The main advantage of the impedance controller is that it allows the FR3 to behave like a programmable mechanical system. By changing the stiffness matrix K_x , the robot can behave either rigidly or compliantly. A high stiffness value produces accurate pose tracking, while a low stiffness value allows safer physical interaction with humans or the environment. The damping matrix D_x is used to reduce oscillations and improve stability during contact.

3.9.2 Pseudo-Algorithm of the Cartesian Admittance Controller

The admittance controller uses the opposite logic of impedance control. Instead of converting position error into a force response, it converts measured external force into

a desired motion. This is useful for hand-guiding and collaborative interaction, because the robot moves according to the force applied by the human or the environment.

The admittance model is written as:

$$M_a \Delta \ddot{x} + D_a \Delta \dot{x} + K_a \Delta x = F_{ext} \quad (3.18)$$

where M_a , D_a , and K_a are the virtual mass, damping, and stiffness of the admittance model. Solving for the virtual acceleration gives:

$$\Delta \ddot{x} = M_a^{-1} (F_{ext} - D_a \Delta \dot{x} - K_a \Delta x) \quad (3.19)$$

The virtual acceleration is integrated to obtain velocity and position corrections. The final desired pose becomes:

$$x_{cmd} = x_{ref} + \Delta x \quad (3.20)$$

The implemented logic of the Cartesian admittance controller can be described as follows:

Step 1: Define the nominal reference pose x_{ref} .

Step 2: Define the admittance parameters: virtual mass M_a , virtual damping D_a , and virtual stiffness K_a .

Step 3: Define the sampling time Δt , which corresponds to the controller loop period.

Step 4: Initialize the virtual displacement:

$$\Delta x \leftarrow 0 \quad (3.21)$$

Step 5: Initialize the virtual velocity:

$$\Delta \dot{x} \leftarrow 0 \quad (3.22)$$

Step 6: Estimate the initial force bias:

$$F_{bias} \quad (3.23)$$

Step 7: Read the current robot state:

$$q, \quad \dot{q}, \quad x, \quad \dot{x}, \quad F_{measured} \quad (3.24)$$

Step 8: Compute the compensated external force:

$$F_{ext} = F_{measured} - F_{bias} \quad (3.25)$$

Step 9: Apply filtering or a small deadband in order to reduce the effect of sensor noise.

Step 10: Compute the virtual acceleration from the admittance model:

$$\Delta\ddot{x} = M_a^{-1} (F_{ext} - D_a\Delta\dot{x} - K_a\Delta x) \quad (3.26)$$

Step 11: Integrate the virtual acceleration to update the virtual velocity:

$$\Delta\dot{x} \leftarrow \Delta\dot{x} + \Delta\ddot{x}\Delta t \quad (3.27)$$

Step 12: Integrate the virtual velocity to update the virtual displacement:

$$\Delta x \leftarrow \Delta x + \Delta\dot{x}\Delta t \quad (3.28)$$

Step 13: Compute the new desired Cartesian command:

$$x_{cmd} = x_{ref} + \Delta x \quad (3.29)$$

Step 14: Convert the Cartesian command into a joint command using inverse kinematics:

$$q_{cmd} = IK(x_{cmd}) \quad (3.30)$$

Step 15: Send the command q_{cmd} to the robot controller.

Step 16: Save the relevant force, position, and command data to the log file.

Step 17: Repeat the loop until the experiment finishes or a safety stop is triggered.

In this controller, the external force is the input and the motion is the output. If a human pushes the robot, the measured force increases and the admittance model produces a displacement in the direction of the applied force. The parameters M_a , D_a , and K_a determine how easily the robot moves. A low stiffness and low virtual mass make the robot easier to guide, while higher values make the robot more resistant to external forces.

3.9.3 Pseudo-Algorithm of the Hybrid Force/Position Controller

The hybrid force/position controller separates the task into position-controlled directions and force-controlled directions. This is especially useful for surface-contact tasks.

For example, when the robot slides an object on a table, the x and y directions can be controlled by position, while the z direction can be controlled by force.

The Cartesian wrench command is constructed from two parts. The first part comes from position control in the horizontal directions:

$$F_x = K_{xy}(x_d - x) - D_{xy}\dot{x} \quad (3.31)$$

$$F_y = K_{xy}(y_d - y) - D_{xy}\dot{y} \quad (3.32)$$

The second part comes from force control in the vertical direction:

$$e_F = F_{z,target} - F_{z,contact} \quad (3.33)$$

$$F_z = K_f e_F - D_f \dot{z} \quad (3.34)$$

The resulting Cartesian wrench is:

$$W_{cmd} = \begin{bmatrix} F_x \\ F_y \\ F_z \\ M_x \\ M_y \\ M_z \end{bmatrix} \quad (3.35)$$

and the torque command is:

$$\tau_{cmd} = J(q)^T W_{cmd} + C(q, \dot{q}) \quad (3.36)$$

The implemented logic of the hybrid force/position controller can be described as follows:

Step 1: Define the desired planar trajectory $x_d(t)$ and $y_d(t)$.

Step 2: Define the desired normal contact force:

$$F_{z,target} \quad (3.37)$$

Step 3: Define the planar position-control gains K_{xy} and D_{xy} .

Step 4: Define the force-control gain K_f and vertical damping gain D_f .

Step 5: Define the contact-force sign coefficient s_c , which is selected experimentally according to the sign convention of the measured vertical force.

Step 6: Move the robot safely to the initial contact or pre-contact pose.

Step 7: Estimate the vertical force bias:

$$F_{z,bias} = \frac{1}{N} \sum_{i=1}^N F_{z,raw,i} \quad (3.38)$$

Step 8: Read the current robot state:

$$q, \quad \dot{q}, \quad x, \quad y, \quad z, \quad \dot{x}, \quad \dot{y}, \quad \dot{z}, \quad F_{z,raw} \quad (3.39)$$

Step 9: Compute the robot Jacobian:

$$J(q) \quad (3.40)$$

Step 10: Compute Coriolis compensation:

$$C(q, \dot{q}) \quad (3.41)$$

Step 11: Compute the compensated vertical force:

$$F_{z,\Delta} = F_{z,raw} - F_{z,bias} \quad (3.42)$$

Step 12: Compute the interpreted contact force:

$$F_{z,contact} = s_c F_{z,\Delta} \quad (3.43)$$

Step 13: Compute the planar position errors:

$$e_x = x_d - x \quad (3.44)$$

$$e_y = y_d - y \quad (3.45)$$

Step 14: Compute the position-control force components:

$$F_x = K_{xy} e_x - D_{xy} \dot{x} \quad (3.46)$$

$$F_y = K_{xy} e_y - D_{xy} \dot{y} \quad (3.47)$$

Step 15: Compute the force error in the vertical direction:

$$e_F = F_{z,target} - F_{z,contact} \quad (3.48)$$

Step 16: Compute the vertical force command:

$$F_z = K_f e_F - D_f \dot{z} \quad (3.49)$$

Step 17: Apply a vertical safety wall if needed:

$$\text{if } z < z_{min}, \quad F_z \leftarrow F_z + K_{wall}(z_{min} - z) \quad (3.50)$$

Step 18: Construct the commanded Cartesian wrench:

$$W_{cmd} = \begin{bmatrix} F_x \\ F_y \\ F_z \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (3.51)$$

Step 19: Convert the Cartesian wrench into joint torques:

$$\tau_{raw} = J(q)^T W_{cmd} + C(q, \dot{q}) \quad (3.52)$$

Step 20: Apply torque-rate saturation:

$$\Delta\tau = \text{clip}(\tau_{raw} - \tau_{prev}, -\Delta\tau_{\max}, \Delta\tau_{\max}) \quad (3.53)$$

$$\tau_{cmd} = \tau_{prev} + \Delta\tau \quad (3.54)$$

Step 21: Send the final torque command τ_{cmd} to the FR3 through the Franka Control Interface.

Step 22: Save position, force, and torque data to the log file.

Step 23: Update the previous torque command:

$$\tau_{prev} \leftarrow \tau_{cmd} \quad (3.55)$$

Step 24: Repeat the loop until the surface-transfer task finishes or a safety stop is triggered.

The hybrid controller is suitable for contact tasks because it allows the robot to control different task-space directions in different ways. In the surface-transfer experiment, the horizontal directions were used for trajectory tracking, while the vertical direction was used for force regulation. This allowed the robot to slide the object along the table while maintaining contact force.

3.9.4 Comparison of the Three Algorithms

The three algorithms differ in the way they relate motion and force.

The impedance controller receives a desired pose and generates a force or torque response when the robot deviates from this pose. Therefore, it can be interpreted as:

$$\text{motion error} \rightarrow \text{force response} \quad (3.56)$$

The admittance controller receives an external force and generates a motion response. Therefore, it can be interpreted as:

$$\text{external force} \rightarrow \text{motion command} \quad (3.57)$$

The hybrid force/position controller separates the task directions. Some directions are controlled by position, while other directions are controlled by force. Therefore, it can be interpreted as:

$$\text{position control in selected axes} + \text{force control in selected axes} \quad (3.58)$$

In this thesis, the impedance controller was mainly used to achieve compliant behavior, the admittance controller was used to study force-driven motion and hand-guiding behavior, and the hybrid controller was used for surface-contact tasks where the robot had to move along a surface while regulating the normal force.

Chapter 4

Hardware/Technical specifications

4.1 Hardware and Technical Specifications

This section summarizes the main hardware characteristics, configuration parameters, and operational limits of the Franka Research 3 platform used in this thesis. These specifications are important because every controller implemented in this work must respect the physical limits of the robot, the gripper, the communication interface, and the safety system. In force-control experiments, these limits are especially important because contact with the environment can create fast variations in force, torque, and velocity.

ARM				
Degrees of freedom	7		Interfaces	• Ethernet (TCP/IP) for visual intuitive programming with Desk • Safety-rated inputs for emergency stop and external enabling device • 2 configurable safety-rated inputs for safeguards or other protective devices (OSSD devices via external OSSD converter connectable) • Hardware prepared for: 2x DI & 2x DO (24V, isolated, EN 61131-2 type 3 characteristics, 100 Hz sampling rate) • Control connector • Connector for end effector
Payload	3 kg			
Maximum reach	855 mm			
Force/Torque sensing	Link-side torque sensor in all 7 axes			
Joint position limits	A1, A3	-166° / 166°		
	A2	-105°/105°		
	A4	-176°/-7°		
	A5	-165°/165°		
	A6	25°/265°		
	A7	-175°/175°		
Mounting flange	DIN ISO 9409-1-A50		User Interfaces at the Arm's Pilot Grip	• Integrated safety-rated guiding enabling switch • Guiding button • Guiding mode button
Installation position	Upright			
Weight	~17,8 kg			
Protection rating	IP40			
Ambieant temperature	+5°C to +45°C		User Interfaces at the Arm's Pilot Disc	• Status light • Pilot mode selector • Arrow keys, teach, confirm, delete
Air humidity	20-80%, non-condensing			
CONTROL			PERFORMANCE	
Control size (19")	483 x 355 x 89 mm (L x W x H)		Motion	
Supply voltage	100-240 V AC		Joint velocity limits	A1-A4: 150°/s
Mains frequency	50-60 Hz			A5-A7: 301°/s (with FCI use A6: 239°/s)
Power consumption	~ 80 W		Cartesian velocity limits	up to 2 m/s end effector speed
Active power factor correction (PFC)	yes		Pose repeatability ¹	< ±0,1 mm (ISO9283)
Weight	~ 7 kg		Interaction	
Protection rating	IP20		Guiding force	~ 2,5 N
Ambient temperature	+5 °C to +45 °C		Adjustable translational stiffness	10-3000 N/m
Air humidity	20-80 %, non-condensing		Adjustable rotational stiffness	1-300 Nm/rad
Permitted mounting orientation	horizontal		Monitored signals	joint position, velocity, torque cartesian position, force

Figure 4.1: Technical specifications of the Franka Research 3 arm, control unit, and performance characteristics. The table summarizes the main hardware properties of the robot, including its seven degrees of freedom, payload, reach, joint torque sensing, joint position limits, velocity limits, Cartesian speed limit, stiffness range, and monitored signals.

4.1.1 FR3 Arm Hardware Overview

The Franka Research 3 is a seven-degree-of-freedom collaborative robotic manipulator. All seven joints are revolute, which gives the robot a redundant anthropomorphic structure. The additional degree of freedom allows the robot to modify its internal posture while maintaining the same end-effector pose. This is useful for avoiding joint limits, improving manipulability, avoiding obstacles, and increasing safety during human–robot interaction.

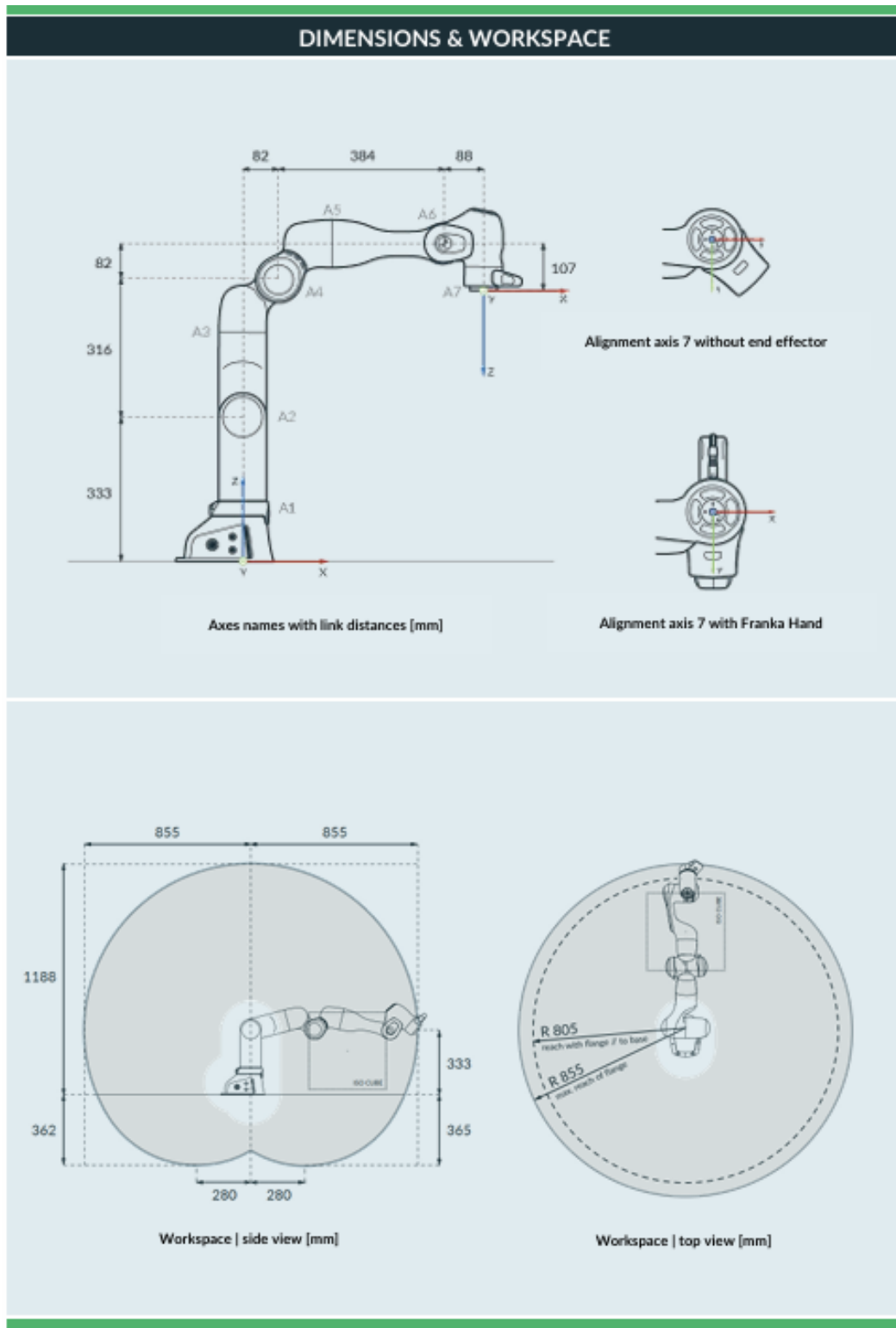


Figure 4.2: Dimensions and workspace of the Franka Research 3. The figure presents the link distances, axis naming, axis-7 alignment with and without the Franka Hand, and the reachable workspace from side and top views.

The most important technical characteristics of the FR3 arm are summarized in Table 4.1.

Table 4.1: Main technical specifications of the Franka Research 3 arm.

Parameter	Value
Degrees of freedom	7 revolute joints
Nominal payload	3 kg
Maximum reach	855 mm
Arm weight	approximately 17.8 kg
Force/torque sensing	link-side torque sensors in all 7 joints
Mounting flange	DIN ISO 9409-1-A50
Installation position	upright installation
Protection rating	IP40
Ambient temperature	+5°C to +45°C
Relative humidity	20–80%, non-condensing
Pose repeatability	$< \pm 0.1$ mm
Research interface	Franka Control Interface at 1 kHz

The integrated link-side torque sensors are one of the most important hardware features for this thesis. They allow the robot to estimate external interaction torques and wrenches without necessarily requiring an additional wrist-mounted force/torque sensor. This makes the FR3 suitable for impedance control, admittance control, hybrid force/position control, hand-guiding, and contact detection.

4.1.2 Joint Position Limits

The joint position limits define the allowed angular range of each joint. These limits must be respected by all joint-space trajectories, inverse-kinematics solutions, null-space motions, and recovery motions. If a command drives the robot close to or beyond a joint limit, the safety system may stop the robot.

The joint position limits of the FR3 are shown in Table 4.2. The values are given both in degrees and radians.

Table 4.2: Joint position limits of the Franka Research 3.

Joint	Limit in degrees	Limit in radians
A1	-166° to 166°	-2.897 to 2.897
A2	-105° to 105°	-1.833 to 1.833
A3	-166° to 166°	-2.897 to 2.897
A4	-176° to -7°	-3.072 to -0.122
A5	-165° to 165°	-2.880 to 2.880
A6	25° to 265°	0.436 to 4.625
A7	-175° to 175°	-3.054 to 3.054

For the experiments of this thesis, the controller was designed to avoid operating very close to these limits. A practical safety margin can be expressed as:

$$q_{min,i} + q_{margin} \leq q_i \leq q_{max,i} - q_{margin} \quad (4.1)$$

where q_{margin} is a conservative joint-limit margin. Keeping the joints away from their limits is useful because it reduces the probability of safety stops, improves inverse-kinematics stability, and gives the null-space controller more freedom to modify the posture of the robot.

4.1.3 Joint Velocity and Cartesian Speed Limits

The maximum joint velocity limits define how fast each joint is allowed to move. These limits are important for trajectory generation, inverse kinematics, admittance control, and any controller that produces joint commands. If the controller generates commands with excessive velocity, the robot may trigger a safety stop or a reflex.

The nominal joint velocity limits are shown in Table 4.3.

Table 4.3: Joint velocity limits of the Franka Research 3.

Joints	Limit in degrees/s	Limit in radians/s
A1–A4	$150^\circ/s$	2.618 rad/s
A5–A7	$301^\circ/s$	5.253 rad/s
A6 with FCI use	$239^\circ/s$	4.171 rad/s

The maximum Cartesian end-effector speed is up to:

$$v_{cart,max} = 2 \text{ m/s} \quad (4.2)$$

However, the experiments in this thesis used much lower speeds than the theoretical maximum. This was necessary because force-control and contact experiments are more sensitive than free-space motions. In practice, the commanded velocities were kept conservative, especially during approach, contact, pressing, hand-guiding, and surface-transfer tasks.

A conservative velocity scaling rule can be written as:

$$|\dot{q}_{cmd,i}| \leq \rho_v \dot{q}_{max,i} \quad (4.3)$$

where ρ_v is a velocity scaling coefficient with:

$$0 < \rho_v < 1 \quad (4.4)$$

For initial real-robot tests, a small value of ρ_v is preferable. This reduces the risk of abrupt motion, velocity-limit violation, and unstable contact behavior.

4.1.4 Control and Interaction Performance

The FR3 is designed for collaborative operation and force-sensitive interaction. The robot provides adjustable Cartesian stiffness and monitored signals that are useful for interaction control. The most relevant control and interaction values are summarized in Table 4.4.

Interfaces	• Ethernet (TCP/IP) for internet and/or shop-floor connection • power connector IEC 60320C14 (V-Lock) • Arm connector	ADD-ONS	
		Fully integrated end effectors	• 2-finger gripper • Vacuum gripper
		Fieldbusses	• Modbus/TCP • OPC UA
EXCLUSIVE			
Research interface	1kHz Franka Control Interface (FCI)		
SAFETY			
Certification			
EN ISO 13849-1:2015 safety of machinery - safety-related parts of control systems		certified by TÜV SÜD Product Service	
Collaborative operation modes			
Safety-rated monitored stop		fully integrated in PLd / Cat. 3	
Hand-guiding		fully integrated in PLd / Cat. 3	
Safety-rated speed and separation monitoring		realizable in combination with external protective devices up to PLd / Cat. 3	
Safety parametrization & validation			
Watchman		user interface to set and validate safety-related parameters	
User management		role based access management	
Safety Functions			
Emergency Stop (x3.1)		PLd / Cat. 3	
External Enabling Device (x4)		PLd / Cat. 3	
Enabling Button		PLd / Cat. 3	
Two configurable safe inputs (x3.2 und x3.3)		PLd / Cat. 3	
SLP-C: Safely limited Cartesian position		PLd / Cat. 3 NOTICE: FCI cannot control the robot while SLP-C is active	
SLS-C: Safely limited Cartesian speed		PLd / Cat. 3 NOTICE: FCI cannot control the robot while SLS-C is active	
SLP-J: Safely limited joint angle		PLd / Cat. 3 NOTICE: FCI cannot control the robot while SLP-J is active	
SLS-J: Safely limited joint speed		PLd / Cat. 3	
SLD: Safely limited distance		PLd / Cat. 3	
SEEPO: Safe End Effector Power off		PLb / Cat. b	
Stopping Functions			
Category 0 stop		PLd / Cat. 3	
Category 1 stop		PLd / Cat. 3	
Category 2 stop		PLd / Cat. 3	
Worst case safe Cartesian position accuracy for stopping functions		50 mm	
Safety values according to EN ISO 13849-1			
PFH of PLd / Cat. 3 safety functions (Probability of Failures per Hour)		< 1 x 10 ⁻⁷	
PFH of PLb / Cat. b safety functions (Probability of Failures per Hour)		< 1 x 10 ⁻⁷	

Figure 4.3: Research interface and safety-related specifications of the Franka Research 3. The figure shows the 1 kHz Franka Control Interface, add-on options, fieldbus support, Watchman safety parametrization, safety-rated collaborative modes, safety functions, stopping functions, and safe Cartesian position accuracy for stopping functions.

Table 4.4: Control and interaction-related specifications.

Parameter	Value
Franka Control Interface frequency	1 kHz
Guiding force	approximately 2.5 N
Adjustable translational stiffness	10–3000 N/m
Adjustable rotational stiffness	1–300 Nm/rad
Monitored joint signals	position, velocity, torque
Monitored Cartesian signals	position, force

The 1 kHz Franka Control Interface is essential for the controllers implemented in this thesis. Impedance control, admittance control, and hybrid force/position control require fast feedback from the robot state. If the communication loop is delayed or interrupted, the robot may stop for safety reasons. For this reason, the control computer must be configured carefully and the controller code must avoid heavy computations inside the real-time loop.

4.1.5 Franka Hand Technical Specifications

The Franka Hand was used as the end-effector for grasping and manipulation experiments. It is an electrical two-finger parallel gripper that connects directly to the robot flange. Power and communication are provided through the X6 end-effector connector, so no external cabling is required for normal use.

Description	Coordinate	Unit	Value
Outer dimension	width	[mm]	63
	length		205
	height		127
Weight		[g]	730
Center of mass of Franka Hand to end effector flange	X	[mm]	0
	Y		9,3
	Z		30,2
Grasping (continuous) force adjustable		[N]	30-70
Travel Range		[mm]	80
Travel Speed (per finger)		[mm/s]	50

Figure 4.4: Mechanical data of the Franka Hand. The table summarizes the gripper dimensions, mass, center of mass relative to the end-effector flange, adjustable continuous grasping force, travel range, and travel speed per finger.

The main specifications of the Franka Hand are shown in Table 4.5.

Table 4.5: Main technical specifications of the Franka Hand.

Parameter	Value
Type	electrical two-finger parallel gripper
Width	63 mm
Length	205 mm
Height	127 mm
Weight	730 g
Travel range	80 mm
Travel speed per finger	50 mm/s
Continuous grasping force	30–70 N
Mechanical interface	DIN ISO 9409-1-A50
Electrical connection	X6 end-effector connector

The center of mass of the Franka Hand relative to the end-effector flange is:

$$p_{CoM}^{hand} = \begin{bmatrix} 0 \\ 9.3 \\ 30.2 \end{bmatrix} \text{ mm} \quad (4.5)$$

The approximate inertia matrix of the closed Franka Hand with respect to the

end-effector flange is:

$$I_{closed} = \begin{bmatrix} 0.0027 & 0 & 0 \\ 0 & 0.0011 & 0 \\ 0 & 0 & 0.0018 \end{bmatrix} \text{ kg m}^2 \quad (4.6)$$

The approximate inertia matrix of the opened Franka Hand is:

$$I_{open} = \begin{bmatrix} 0.0030 & 0 & 0 \\ 0 & 0.0011 & 0 \\ 0 & 0 & 0.0021 \end{bmatrix} \text{ kg m}^2 \quad (4.7)$$

Correct end-effector configuration is important because the robot uses the payload mass, center of mass, and inertia parameters for gravity compensation, collision behavior, and force estimation. If the Franka Hand or an additional object is configured incorrectly, the robot may not compensate gravity correctly. This can cause unexpected motion during guiding, incorrect external force estimates, reduced sensitivity, and degraded tracking behavior.

4.1.6 Payload and End-Effector Configuration

The nominal payload of the FR3 is:

$$m_{payload,max} = 3 \text{ kg} \quad (4.8)$$

This payload includes the gripper, any additional tool, and the object being manipulated. Therefore, when the Franka Hand is mounted, the available payload for grasped objects is reduced by the mass of the hand:

$$m_{remaining} = m_{payload,max} - m_{hand} \quad (4.9)$$

Using $m_{hand} = 0.730 \text{ kg}$, the approximate remaining payload becomes:

$$m_{remaining} = 3.000 - 0.730 = 2.270 \text{ kg} \quad (4.10)$$

In this thesis, the manipulated objects were lightweight, so the payload limit was not a restrictive factor. However, the end-effector configuration was still important because even a small payload or center-of-mass error can affect force estimation and contact behavior.

For every experiment with an end-effector, the following parameters should be checked:

- mass of the mounted end-effector,

- center of mass of the end-effector,
- inertia matrix of the end-effector,
- mass of the grasped object,
- center of mass of the grasped object,
- total payload below the maximum allowed value,
- correct tool frame and contact point.

4.1.7 Mounting and Installation Requirements

The FR3 arm must be mounted on a stable, level, non-moving platform. This is important because the robot uses sensitive torque sensing and dynamic control. If the base moves or vibrates, the robot state estimation and force-control behavior can be affected.

Classification of spaces

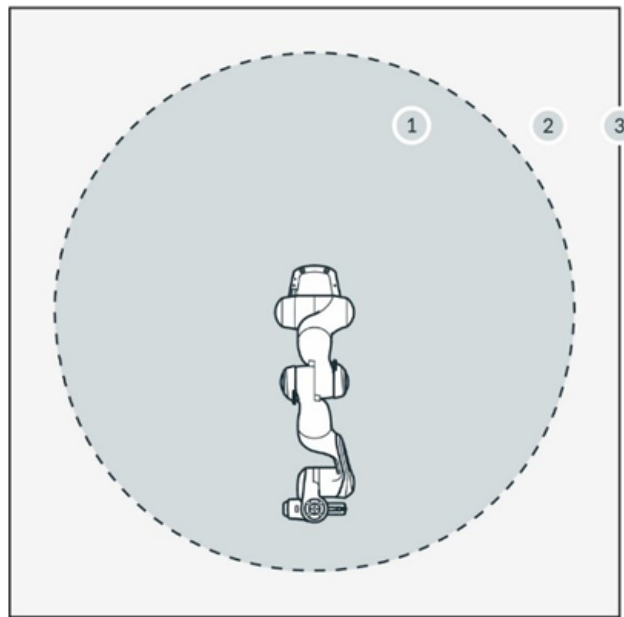


Fig. 10.12: Classification of spaces

1	Maximum workspace	3	Perimeter safeguarding
2	Safeguarded space		

Figure 4.5: Classification of the installation space around the Franka Research 3. The figure distinguishes between the maximum workspace, the safeguarded space, and the perimeter safeguarding, which are essential for planning safe real-robot experiments.

The arm must be installed upright, with the base horizontal to the ground. The maximum permitted tilt angle of the platform is approximately:

$$\theta_{tilt,max} = 0.1^\circ \quad (4.11)$$

The mounting base must be able to withstand the static and dynamic loads generated by the robot. The relevant mounting-load requirements are summarized in Table 4.6.

Table 4.6: Required support loads for the FR3 mounting base.

Load type	Required support value
Tilting torque	280 Nm
Torque around vertical axis	190 Nm
Horizontal force	300 N
Vertical force	410 N

The arm is mounted with four M8 screws. The recommended tightening torque for the base screws is:

$$T_{base} = 30 \text{ Nm} \quad (4.12)$$

The Franka Hand is mounted to the end-effector flange using two M6x12 DIN 7984 screws. The tightening torque for the Franka Hand mounting screws is:

$$T_{hand} = 5 \text{ Nm} \quad (4.13)$$

The functional earth connection should also be connected according to the installation instructions. This is required for meeting electromagnetic compatibility requirements and for stable operation in the laboratory environment.

4.1.8 Network and FCI Configuration

The Franka Control Interface is used for real-time control from an external Linux workstation. For FCI operation, the workstation must be connected to the C2 shop-floor network port of the Franka Control unit. The X5 robot network port at the robot base is used for robot-network access and initial configuration, but it is not the correct interface for FCI control.

The network configuration used in this thesis is summarized in Table 4.7.

Table 4.7: Network configuration used for FCI communication.

Device	Interface purpose	IP address
Linux workstation	external FCI computer	172.16.0.1/24
FR3 Control unit	C2 shop-floor network	172.16.0.2/24
Robot base X5 network	robot-network access	192.168.0.1/24

The direct FCI connection was preferred for the real-time control experiments because it reduces network uncertainty. The communication loop must remain stable at 1 kHz. A successful ping only verifies basic IP connectivity; it does not guarantee that the workstation can receive the real-time robot-state packets required by `libfranka`. Therefore, communication was also verified using `echo_robot_state` before executing real robot experiments.

4.1.9 Safety Functions and Practical Limits

The FR3 includes several safety functions that monitor position, speed, standstill, safe inputs, and end-effector power. These functions are configured through Watchman and Desk. They are essential for safe operation, especially during contact-rich tasks and human-robot interaction.

The safety-related functions most relevant to this thesis are:

- **Emergency Stop:** connected through X3.1 and used to stop the robot in a safety-critical situation.
- **External Enabling Device:** connected through X4 and used during test and jog operation.
- **Safely Limited Cartesian Position:** monitors Cartesian position limits of the robot and configured tool spheres.
- **Safely Limited Cartesian Speed:** monitors Cartesian speed of robot points and tool spheres.
- **Safely Limited Joint Position:** internally protects joint angle limits.
- **Safely Limited Joint Speed:** internally monitors joint speed.
- **Safe End Effector Power Off:** switches off the 48 V end-effector power line when required by the safety configuration.

For the experiments of this thesis, additional conservative software limits were also used. These limits were not replacements for the certified safety functions, but they reduced the probability of unstable commands and reflex stops.

The practical software limits included:

- small trajectory amplitudes during the first tests,
- smooth interpolation for all point-to-point motions,
- reduced Cartesian speed during contact approach,
- low initial force targets,
- torque-rate saturation,
- force-bias estimation before contact control,
- vertical safety wall during surface-contact tasks,
- continuous monitoring of external torque and wrench estimates,
- immediate availability of the emergency stop.

A typical torque-rate saturation rule used in torque-control experiments is:

$$\Delta\tau_i = \text{clip}(\tau_{raw,i} - \tau_{prev,i}, -\Delta\tau_{max}, \Delta\tau_{max}) \quad (4.14)$$

$$\tau_{cmd,i} = \tau_{prev,i} + \Delta\tau_i \quad (4.15)$$

This prevents sudden jumps in commanded torque and improves the stability of the real-time control loop.

4.1.10 Importance of Technical Limits for the Controllers

The hardware and technical limits of the FR3 directly affect the controller design. The impedance controller must respect torque-rate limits and avoid excessive stiffness. The admittance controller must limit the velocity and displacement produced by external forces. The hybrid force/position controller must regulate contact force without commanding unsafe vertical motion. In all cases, the controller must remain inside the robot’s joint limits, velocity limits, payload limits, and safety configuration.

For this reason, the implemented control framework follows a conservative design philosophy:

$$\text{safe motion} + \text{stable contact} + \text{bounded force} + \text{bounded torque} \quad (4.16)$$

The goal is not only to make the robot execute the desired task, but also to ensure that the task is executed in a way that is repeatable, safe, and suitable for physical human-robot interaction.

Chapter 5

Simulation Results

5.1 The Robot Operating System

The Robot Operating System (ROS) is a powerful and flexible software platform for building robotic applications. It consists of libraries and tools that provide a wide range of functionality, including hardware abstraction, low-level device control, and high-level robotic behaviors. ROS is modular, allowing developers to create and integrate new applications by building on existing packages.

ROS provides drivers and algorithms for many hardware platforms and sensor types, making it easier to connect and control robotic devices. It also includes developer tools for visualization, debugging, and performance analysis. One of the main advantages of ROS is that it is open source, which makes it suitable for collaborative robotics projects where researchers and developers can share and extend each other's work.

In ROS, the basic computational units are called nodes. Each node is responsible for a specific task. In a robotic system, multiple nodes can cooperate to perform larger distributed tasks such as motor control, perception, sensor filtering, and high-level decision making. This modular architecture makes it possible to modify one part of the system without rewriting the entire code base.

Nodes can be implemented in different programming languages, such as C++ and Python, and still communicate with each other. C++ is often preferred for real-time or high-performance applications because it is compiled and typically faster than interpreted languages. For the experiments in this thesis, high real-time performance was important, especially for the implementation of controllers that interact with the Franka Control Interface.

5.2 Gazebo Simulator

Gazebo is a critical tool for pre-experimental validation. It allows force-control and obstacle-recognition algorithms to be tested in a risk-free physics environment before being deployed on real hardware. Gazebo is an open-source 3D robotics simulator that integrates with ROS. Unlike a simple animation tool, Gazebo uses physics engines, such as ODE or Bullet, to compute forces, gravity, and collisions.

For this thesis, Gazebo is useful for several reasons:

- **Torque simulation:** Gazebo can simulate the joint torque behavior of the FR3. The same dynamic terms $M(q)$, $C(q, \dot{q})$, and $g(q)$ can be used to test whether the simulated robot behaves correctly under gravity compensation and impedance control.
- **Contact dynamics:** Gazebo simulates contact through a physics model. When the virtual FR3 touches a virtual table, the simulator computes reaction forces. These forces can be used to tune the impedance-control gains K and D and to avoid bouncing or instability.
- **Obstacle recognition:** the simulation world can be populated with objects such as boxes, spheres, or human-like models. This allows testing of null-space obstacle avoidance while the end-effector remains on its primary task.

To simulate the FR3 in Gazebo, the following components are required:

1. **URDF/Xacro model:** an XML-based description of the robot's kinematics, visual meshes, collision geometry, masses, and inertial properties.
2. **ros_control:** the bridge between the controller code and the simulated robot. For this thesis, an effort-based interface is required so that torque commands can be sent instead of only position commands.
3. **Gazebo plugins:** plugins can be used to simulate sensors, controllers, and external perception devices such as RGB-D cameras.

Before physical deployment, the algorithms were validated in the Gazebo simulator. The robot was modeled using a URDF file that includes the kinematic chain and the inertial properties of each link. To simulate force control, the `gazebo_ros_control` plugin was used with an effort joint interface. This allowed the simulator to accept torque commands τ_{cmd} and return simulated joint-torque data, creating a digital-twin testing environment.

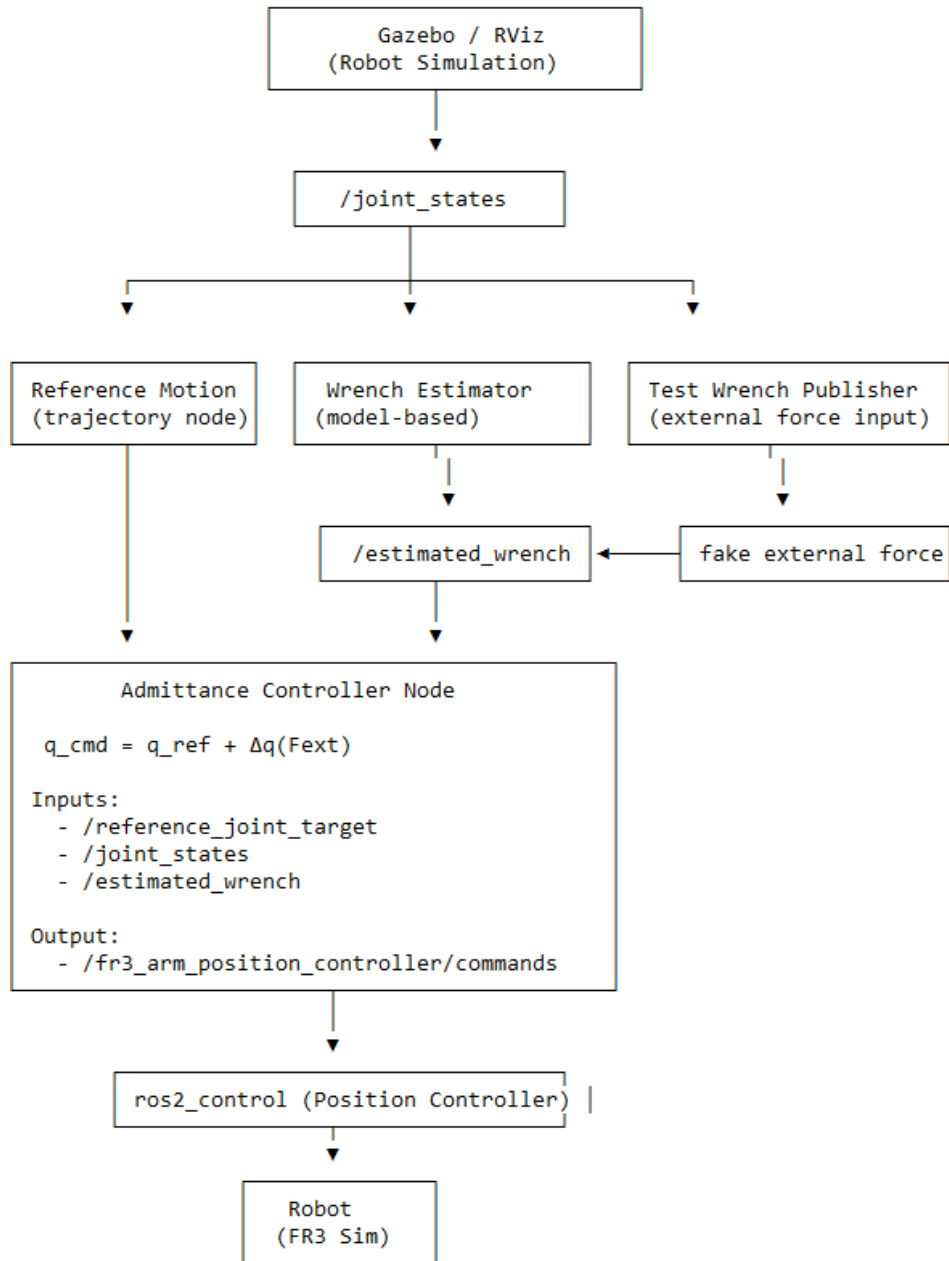


Figure 5.1: ROS 2 and Gazebo simulation architecture of the admittance controller. The diagram shows how the simulated robot state is published through `/joint_states`, processed by the wrench estimator and the admittance controller, and finally converted into position commands for the FR3 simulated robot through `ros2_control`.

As shown in Figure 5.1, the admittance-control architecture was first designed and tested in simulation before being transferred to the real FR3 robot. The simulated robot runs in Gazebo and is visualized through RViz. The robot publishes its joint state through the `/joint_states` topic, which is used by the reference motion node, the wrench estimator, and the admittance controller.

The reference motion node generates the desired nominal joint trajectory, while the wrench estimator computes an estimated external wrench using the current robot state

and model-based information. In addition, a test wrench publisher can inject a fake external force into the system. This is useful during simulation because it allows the admittance controller to be tested without requiring real physical contact.

The admittance controller receives three main inputs: the reference joint target, the current joint state, and the estimated wrench. Based on the measured or simulated external force, it computes a compliant correction $\Delta q(F_{ext})$ and produces the commanded joint configuration:

$$q_{cmd} = q_{ref} + \Delta q(F_{ext})$$

The resulting command is sent to the `/fr3_arm_position_controller/commands` topic. The `ros2_control` position controller then applies the command to the simulated FR3 robot. This architecture makes it possible to validate the admittance-control logic in a safe simulation environment before executing similar ideas on the physical FR3 manipulator.

Chapter 6

Experimental Results

6.1 Franka Control Interface

The Franka Control Interface (FCI) allows custom controllers to run on an external real-time capable Linux computer. It provides access to robot-state data and allows torque commands to be sent at high frequency. This interface is essential for the work of this thesis because impedance control requires fast feedback from measured joint states, estimated external torques, Cartesian poses, and force-related quantities.

The real-time communication loop is sensitive to latency. If the loop misses deadlines, the robot may stop for safety reasons. For this reason, the control computer was configured with a real-time Linux kernel and a direct Ethernet connection to the robot controller. The controller node was designed to avoid heavy computations inside the hard real-time loop, ensuring stable communication and safe execution of torque commands.

The FCI was used as the main interface for the real-robot experiments of this thesis. Through this interface, the implemented controllers could read the robot state, estimate interaction forces, send joint torque commands, and log experimental data for post-processing. The experiments were designed to evaluate the behavior of the FR3 under free-space motion, contact with the environment, external human disturbances, compliant hand-guiding, object manipulation, and variable contact conditions.

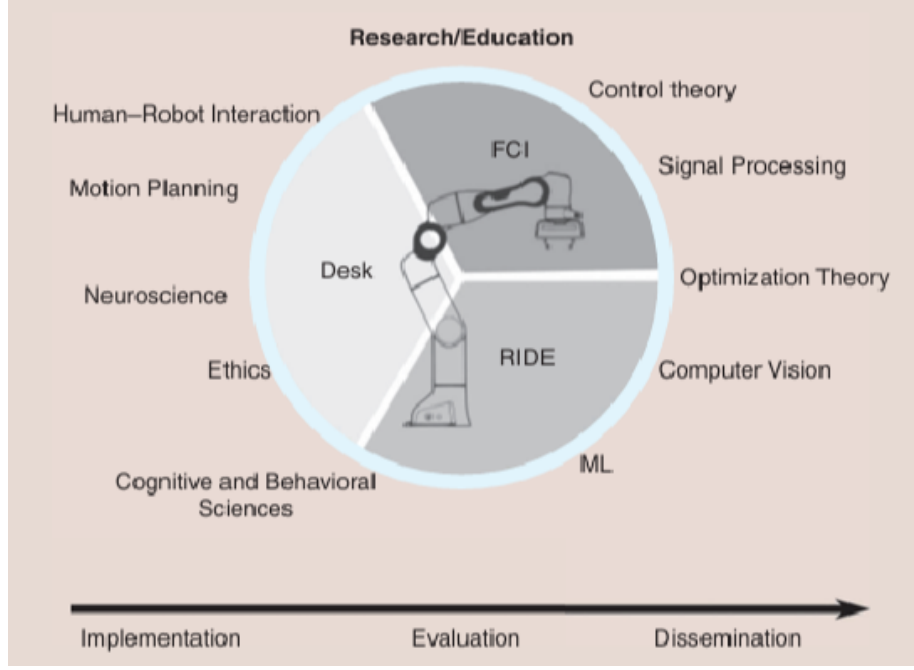


Figure 6.1: Franka Research 3 as a research and education platform. The figure illustrates the role of the FR3 ecosystem, including Desk, FCI, and RIDE, and shows how the platform supports several research fields such as human-robot interaction, control theory, motion planning, computer vision, machine learning, signal processing, and optimization.

The figure above presents the Franka Research 3 as a multidisciplinary research and education platform. The central part of the figure shows the FR3 robot together with the main software and control components of the Franka ecosystem. Desk provides the user interface for robot monitoring, safety configuration, unlocking, recovery, and activation of the robot. The Franka Control Interface (FCI) provides the low-level communication layer that allows external real-time controllers to access robot-state data and send commands to the robot. RIDE represents the higher-level development and execution environment provided by the Franka ecosystem.

The surrounding terms in the figure show that the FR3 can be used in many different research areas. In the context of this thesis, the most relevant areas are human-robot interaction, control theory, motion planning, signal processing, optimization, computer vision, and machine learning. Human-robot interaction is directly related to the goal of safe physical contact between the robot and the user. Control theory is necessary for the design of impedance, admittance, and hybrid force/position controllers. Signal processing is useful for filtering torque and force estimates. Optimization theory is related to redundancy resolution, null-space control, and trajectory generation. Computer vision and machine learning can be used in future extensions for object detection, human intention prediction, and adaptive control.

The horizontal axis at the bottom of the figure indicates the typical research work-

flow from implementation to evaluation and dissemination. This is also the structure followed in this thesis. First, the control algorithms were implemented using libfranka, ROS 2, MoveIt, and Gazebo. Then, the methods were evaluated through simulation and real-robot experiments on the FR3. Finally, the results were analyzed and documented in order to assess the suitability of the FR3 for force-control and human-interaction tasks.

6.1.1 Experimental Campaign

The experimental campaign was organized in a progressive way. The first experiments focused on simple and safe free-space motion, while later experiments introduced contact, force regulation, human interaction, and different environmental materials. This gradual structure was necessary because force-control experiments are more sensitive than pure position-control experiments. Before testing contact-rich tasks, the robot motion, network communication, controller stability, and safety limits had to be validated.

For every experiment, the following quantities were considered important for evaluation:

- joint positions and joint velocities,
- end-effector Cartesian position,
- desired and actual trajectory,
- position tracking error,
- measured or estimated external wrench,
- external joint torques,
- commanded joint torques,
- contact force, when contact was present,
- controller stability and safety behavior.

The experiments were executed using conservative limits at the beginning. The robot was always monitored through Desk, and the emergency stop was kept accessible. The purpose of the experiments was not only to demonstrate that the robot could move, but also to evaluate how the implemented controllers behaved when the robot interacted with its environment or with a human operator.

6.1.2 Experiment 1: Baseline Free-Space Joint Trajectory

The first experiment was a baseline free-space joint trajectory. The purpose of this experiment was to verify that the robot could execute a smooth joint-space motion without contact with the environment. This experiment was used as the reference case for the rest of the experimental campaign.

In this experiment, a desired joint trajectory was generated for one or more joints of the FR3. The commanded trajectory was intentionally small and smooth in order to avoid sudden accelerations or velocity discontinuities. The robot moved in free space, without touching any object or surface.

The controller objective was to minimize the joint tracking error:

$$e_q(t) = q_{ref}(t) - q(t)$$

where $q_{ref}(t)$ is the desired joint trajectory and $q(t)$ is the measured joint position. A simple joint-space control law can be written as:

$$\tau_{cmd} = K_p(q_{ref} - q) + K_d(\dot{q}_{ref} - \dot{q})$$

where K_p and K_d are the proportional and derivative gain matrices.

The main goal of this experiment was to confirm that the FCI communication, controller loop, and torque command pipeline were working correctly. Since there was no intentional contact, the measured external forces and external torques were expected to remain small. Any large external torque estimate during this experiment would indicate a possible modeling error, incorrect payload configuration, or unexpected contact.

The logged data from this experiment were used to evaluate the quality of the trajectory tracking. Important plots include actual versus desired joint position, joint tracking error, commanded torque, and external torque norm. This experiment provided the baseline behavior of the FR3 before introducing contact forces.

6.1.3 Experiment 2: Contact Approach with a Soft Object

The second experiment introduced contact with a soft object. The purpose was to evaluate how the robot behaves when it approaches and touches a compliant environment. A soft object was selected because it reduces the risk of high contact forces and allows the first contact experiment to be performed safely.

In this experiment, the robot moved slowly toward the object along a controlled Cartesian direction. The motion was performed using a low-speed Cartesian trajectory. When the end-effector touched the soft object, the external wrench estimate was monitored. The most important variable was the contact force along the approach direction.

The contact force can be represented as:

$$F_{contact} = F_{measured} - F_{bias}$$

where $F_{measured}$ is the raw force estimate and F_{bias} is the estimated force offset before contact.

The objective of this experiment was not to regulate a precise force yet, but to observe the transition from free-space motion to contact. The expected behavior was a gradual increase in measured force due to the compliance of the soft object. Unlike contact with a rigid surface, contact with a soft object should not produce a sudden force spike.

This experiment was important because it validated the force-sensing pipeline and helped determine the correct force sign convention. It also helped tune safe velocity limits and basic contact thresholds before performing stronger force-control experiments.

6.1.4 Experiment 3: Constant Force Pressing Experiment

The third experiment was a constant force pressing experiment. The purpose was to test whether the robot could maintain a desired normal force against a surface or object. This experiment is directly related to practical tasks such as polishing, wiping, pressing, and surface processing.

In this experiment, the robot first approached the surface slowly. After contact was detected, the controller attempted to regulate the normal contact force to a desired value:

$$F_{target}$$

The force error was computed as:

$$e_F = F_{target} - F_{contact}$$

where $F_{contact}$ is the measured compensated contact force. A simple force feedback law can be written as:

$$F_{cmd} = K_f e_F - D_f \dot{z}$$

where K_f is the force-control gain and D_f is a damping term used to reduce oscillations during contact. The commanded Cartesian force was then transformed into joint torques using the Jacobian transpose:

$$\tau_{cmd} = J(q)^T W_{cmd} + C(q, \dot{q})$$

where W_{cmd} is the commanded Cartesian wrench and $C(q, \dot{q})$ is the Coriolis compensation term.

The expected result was that the measured force should converge close to the target force and remain stable. If the force oscillated or exceeded the target by a large amount, the controller gains had to be reduced or the damping had to be increased. This experiment was one of the most important tests for evaluating the quality of the implemented force-control strategy.

6.1.5 Experiment 4: Human Push and Disturbance Rejection

The fourth experiment tested the response of the robot to external human disturbances. The purpose was to evaluate whether the robot could remain stable and safe when a human applies a push to the arm or end-effector.

In this experiment, the robot was commanded to hold a desired Cartesian pose or follow a slow reference trajectory. During execution, a human applied a small external disturbance. The controller response was evaluated by observing the displacement of the end-effector, the measured external wrench, and the recovery behavior after the disturbance was removed.

For an impedance controller, the robot response depends mainly on the selected stiffness and damping parameters:

$$F_{cmd} = K(x_{ref} - x) + D(\dot{x}_{ref} - \dot{x})$$

A high stiffness value makes the robot reject the disturbance more strongly, while a lower stiffness value allows the robot to move more compliantly. Therefore, this experiment demonstrates the trade-off between disturbance rejection and safe physical compliance.

The expected behavior depends on the selected control mode. With a stiff configuration, the robot should return close to the reference position after the disturbance. With a soft configuration, the robot should allow larger displacement and feel safer during physical interaction. The external torque estimates were also monitored to verify that the disturbance was detected by the robot sensors.

This experiment is important for human-robot interaction because collaborative robots must not behave as perfectly rigid systems when humans touch them. A safe controller should reject disturbances when necessary, but it should also avoid excessive resistance that could endanger the human operator.

6.1.6 Experiment 5: Hand-Guiding and Lead-Through Experiment

The fifth experiment evaluated hand-guiding, also known as lead-through control. The purpose was to allow a human operator to physically guide the robot by applying forces to the end-effector or robot body.

In hand-guiding, the robot should not strongly resist the human. Instead, it should behave as if it has low virtual stiffness and suitable damping. This behavior can be achieved using admittance control or low-stiffness impedance control.

In an admittance-based approach, the measured external force is used as the input of a virtual mass-spring-damper model:

$$M\Delta\ddot{x} + D\Delta\dot{x} + K\Delta x = F_{ext}$$

Solving for the virtual acceleration gives:

$$\Delta\ddot{x} = M^{-1}(F_{ext} - D\Delta\dot{x} - K\Delta x)$$

The virtual acceleration is integrated to obtain a desired velocity and a desired position update. The robot then moves according to the human-applied force.

The expected behavior is that when the human pushes the robot, the end-effector moves smoothly in the direction of the applied force. When the human stops pushing, the robot should stop without oscillation. If the virtual damping is too low, the robot may continue moving or oscillate. If the virtual stiffness is too high, the robot may feel difficult to guide.

This experiment is useful for demonstrating safe physical human-robot interaction. It also shows how the FR3 can be used in collaborative tasks where the human teaches or guides the robot through physical contact.

6.1.7 Experiment 6: Pick and Place Demonstration

The sixth experiment was a pick and place demonstration. The purpose was to combine Cartesian motion, gripper control, and basic object manipulation in a complete task.

The task sequence was:

1. move to a pre-grasp pose,
2. open the gripper,
3. approach the object,
4. close the gripper and grasp the object,
5. lift the object,

6. move to the target pose,
7. release the object,
8. return to a safe pose.

The pick and place experiment was important because it tested the integration of motion generation and gripper control. Although this task does not necessarily require continuous force control, it provides the foundation for more advanced manipulation tasks. In particular, it validates whether the robot can approach an object safely, grasp it, and move it without losing stability.

For the FR3, this experiment also helped calibrate practical parameters such as grasp height, gripper width, grasping force, approach speed, and safe lifting distance. If the gripper did not confirm the grasp, the object width, grasp height, or grasping force had to be adjusted.

This experiment was also a useful preparation step for the final surface-transfer task, because the robot first had to grasp an object before applying contact force and sliding it on a surface.

6.1.8 Experiment 7: Variable Stiffness Impedance Experiment

The seventh experiment tested variable stiffness impedance control. The purpose was to evaluate how the robot behavior changes when the stiffness parameter of the impedance controller is modified during or between tasks.

In Cartesian impedance control, the robot behaves like a virtual spring-damper system:

$$F_{cmd} = K(x_{ref} - x) + D(\dot{x}_{ref} - \dot{x})$$

where K is the Cartesian stiffness matrix and D is the damping matrix. The stiffness matrix determines how strongly the robot resists displacement from the reference pose.

In the variable stiffness experiment, different stiffness values were tested. For example, a low stiffness configuration can be used during human interaction or hand-guiding, while a higher stiffness configuration can be used during accurate positioning or contact-force regulation.

The expected behavior is:

- with low stiffness, the robot feels soft and compliant;
- with high stiffness, the robot tracks the desired pose more strictly;
- with insufficient damping, oscillations may occur;
- with excessive stiffness, contact forces may become too high.

This experiment is important because a fixed stiffness value is not ideal for all tasks. Human interaction, free-space motion, and surface contact require different dynamic behaviors. Variable stiffness control allows the robot to adapt its behavior according to the task phase.

6.1.9 Experiment 8: Physiotherapy Test Experiment

The eighth experiment was a physiotherapy-inspired test. The purpose was to explore how the FR3 could be used in a safe physical interaction scenario where the robot assists or resists human limb motion. This experiment is relevant to rehabilitation robotics, where robots are used to guide, support, or measure human movement.

In this test, the robot was configured to behave compliantly. The human could move the robot end-effector along a slow and controlled path, while the robot measured the interaction force. Depending on the selected controller, the robot could either assist the motion, resist the motion with a controlled impedance, or remain compliant and allow the human to guide it.

A simplified interaction model can be written as:

$$F_{human} = M\Delta\ddot{x} + D\Delta\dot{x} + K\Delta x$$

where F_{human} is the force applied by the human, and M , D , and K define the virtual behavior of the robot.

The expected behavior was that the robot should remain stable, smooth, and safe during physical interaction. The measured force should remain within safe limits, and the robot should not produce sudden movements. This type of experiment is useful for demonstrating the relevance of force control beyond industrial tasks, especially in applications where physical interaction with humans is required.

Because physiotherapy-related experiments involve direct human contact, they require conservative gains, low speed, low force limits, and immediate emergency stop availability. The goal of this thesis experiment was not to develop a complete medical rehabilitation system, but to demonstrate the basic control behavior required for safe assistive interaction.

6.1.10 Experiment 9: Different Contact Materials

The ninth experiment evaluated contact with different materials. The purpose was to study how the force-control behavior changes when the robot interacts with surfaces or objects of different stiffness, damping, and friction.

Different materials can produce very different contact behavior. A soft foam object deforms easily and produces gradual force increase. A rubber surface may introduce

damping and friction. A rigid wooden or plastic surface may produce a faster force increase and higher risk of oscillation. Therefore, testing different materials is important for evaluating the robustness of the controller.

In this experiment, the robot performed the same contact or pressing motion on different materials. For each material, the measured force, force error, position displacement, and torque response were logged. The same desired force target could produce different measured responses depending on the material properties.

The expected behavior was:

- soft materials should produce smoother and slower force increase;
- rigid materials should produce faster force increase and require more careful gain tuning;
- high-friction materials may affect sliding tasks and increase tracking error;
- low-friction materials may cause object slipping during manipulation.

This experiment is important because real-world robot interaction does not occur with only one ideal surface. A robust force-control strategy must remain stable across different environmental conditions. The results from this experiment can help determine whether the selected impedance, admittance, or hybrid force/position gains are suitable for general use or only for a specific surface.

6.1.11 Summary of the FCI Experiments

The experiments performed through the Franka Control Interface demonstrate a gradual transition from simple motion control to advanced physical interaction. The baseline free-space trajectory verified the basic control pipeline. The soft-object approach introduced safe contact detection. The constant force pressing experiment tested force regulation. The human disturbance experiment evaluated safety and compliance under external pushes. The hand-guiding experiment demonstrated lead-through interaction. The pick and place demonstration validated object manipulation. The variable stiffness experiment showed how the robot behavior can be adapted by changing impedance parameters. The physiotherapy-inspired test demonstrated the relevance of compliant control for human-assistive tasks. Finally, the different-materials experiment evaluated the robustness of the controller under different contact conditions.

Together, these experiments provide a complete evaluation framework for force control and human interaction with the FR3 manipulator. They show that the FCI is not only a communication interface, but also a key enabling technology for implementing, testing, and validating real-time interaction controllers on a torque-sensitive collaborative robot.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

This thesis investigated a force-sensitive control framework for the Franka Research 3 manipulator, focusing on safe human-robot interaction. By exploiting the robot’s internal torque sensors and the Franka Control Interface, torque-based controllers were developed. The FR3 is a reliable reference platform for this type of research because of the quality of its sensor data, its torque-control capabilities, and its redundant 7-DOF structure.

The main conclusions of the thesis are the following:

- **Effectiveness of impedance control:** transforming the FR3 into a virtual mass-spring-damper system proved effective for physical interaction. The robot demonstrated dynamic transparency, allowing users to guide the arm with reduced effort while maintaining stability during contact tasks.
- **Proprioceptive sensing:** the FR3’s link-side torque sensors can be used to estimate external forces through torque residuals. This can reduce the need for expensive external wrist force/torque sensors and simplify the system architecture.
- **Redundancy as a safety tool:** null-space projection allows the robot to use its seventh degree of freedom for reactive behaviors. The arm can reconfigure its posture to avoid obstacles without disturbing the primary end-effector task, demonstrating the benefits of 7-DOF anthropomorphic designs in collaborative environments.

7.2 Challenges and Limitations

Despite the successful implementation, several challenges were identified:

- **Friction and model uncertainty:** although the FR3 provides high-quality dynamic models, unmodeled friction, cable effects, and small modeling errors can introduce noise into torque residuals, especially at low velocities.
- **Computational latency:** maintaining a strict 1 kHz control loop requires careful optimization of the ROS software stack, the controller code, and the real-time Linux configuration.

7.3 Future Work

The research presented in this thesis opens several directions for future work:

- **Integration of vision-based AI:** future versions of the system could combine the force-control framework with deep-learning-based vision in order to predict human intentions before physical contact occurs.
- **Variable impedance control:** algorithms that dynamically adjust stiffness K and damping D according to the task context or the user's expertise could further improve comfort and safety.
- **Multi-contact recognition:** the torque observer could be extended to distinguish between multiple simultaneous contacts on different parts of the robot body.

Chapter 8

Bibliography

- [1] N. Hogan, “Impedance Control: An Approach to Manipulation: Part I—Theory,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 107, no. 1, pp. 1–7, Mar. 1985, doi: 10.1115/1.3140702.
- [2] M. H. Raibert and J. J. Craig, “Hybrid Position/Force Control of Manipulators,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 103, no. 2, pp. 126–133, Jun. 1981, doi: 10.1115/1.3139652.
- [3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2009, doi: 10.1007/978-1-84628-642-1.
- [4] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Hoboken, NJ, USA: John Wiley & Sons, 2006.
- [5] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005.
- [6] Franka Robotics, “Franka Control Interface Documentation.” [Online]. Available: <https://frankarobotics.github.io/docs/>. Accessed: May 14, 2026.
- [7] Open Robotics, “ROS Developer Documentation.” [Online]. Available: <https://docs.ros.org/>. Accessed: May 14, 2026.
- [8] Open Robotics, “Gazebo Documentation.” [Online]. Available: <https://gazebo.sim.org/docs/latest/>. Accessed: May 14, 2026.
- [9] R. Tedrake, “Manipulator Control,” *Robotic Manipulation: Perception, Planning, and Control*, Massachusetts Institute of Technology, 2024. [Online]. Available: <https://manipulation.csail.mit.edu/force.html>. Accessed: May 14, 2026.
- [10] J. Haviland and P. Corke, “Manipulator Differential Kinematics: Part I: Kinematics, Velocity, and Applications,” *IEEE Robotics & Automation Magazine*, pp. 2–12, 2023, doi: 10.1109/MRA.2023.3270228.

- [11] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, U.K.: Cambridge University Press, 2017, doi: 10.1017/9781316661239.
- [12] Sapienza University of Rome, Department of Computer, Control, and Management Engineering Antonio Ruberti, “Robotics 1: Written Exam,” Jul. 8, 2024. [Online]. Available: https://www.diag.uniroma1.it/deluca/rob1_en/Write_nExamsRob1/Robotics1_24.07.08.pdf. Accessed: May 14, 2026.
- [13] A. J. Elias and J. T. Wen, “Redundancy Parameterization and Inverse Kinematics of 7-DOF Revolute Manipulators,” *arXiv preprint arXiv:2307.13122*, 2023, doi: 10.48550/arXiv.2307.13122.
- [14] ffall007, “franka_analytical_ik: An Analytical Inverse Kinematics Solver for Franka Emika Panda,” GitHub repository. [Online]. Available: https://github.com/ffall007/franka_analytical_ik. Accessed: May 14, 2026.
- [15] Y. He and S. Liu, “Analytical Inverse Kinematics for Franka Emika Panda—A Geometrical Solver for 7-DOF Manipulators with Unconventional Design,” in *Proc. 9th International Conference on Control, Mechatronics and Automation (ICCMA)*, 2021, doi: 10.1109/ICCMA54375.2021.9646185.
- [16] P. C. Lopez-Custodio, Y. Gong, and L. F. C. Figueredo, “GeoFIK: A Fast and Reliable Geometric Solver for the IK of the Franka Arm Based on Screw Theory Enabling Multiple Redundancy Parameters,” *arXiv preprint arXiv:2503.03992*, 2025.
- [17] S. Kruzic, J. Music, R. Kamnik, and V. Papic, “End-Effector Force and Joint Torque Estimation of a 7-DoF Robotic Manipulator Using Deep Learning,” *Electronics*, vol. 10, no. 23, Art. no. 2963, Nov. 2021, doi: 10.3390/electronics10232963.
- [18] Z. Xie, S. Li, and L. Jin, “A Bi-Criteria Kinematic Strategy for Motion/Force Control of Robotic Manipulator,” *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 4, pp. 5570–5582, 2024, doi: 10.1109/TASE.2023.3313564.
- [19] S. Haddadin, “The Franka Emika Robot: A Standard Platform in Robotics Research [Survey],” *IEEE Robotics & Automation Magazine*, vol. 31, no. 4, pp. 136–148, 2024, doi: 10.1109/MRA.2024.3451788.
- [20] Technische Universitaet Chemnitz, “Publications,” *Neurorobotik, Faculty of Computer Science*. [Online]. Available: <https://www.tu-chemnitz.de/informatik/neurorobotik/pub/>. Accessed: May 14, 2026.

- [21] IRIS Lab, “Forward Kinematics,” *Modeling and Control of Robots*. [Online]. Available: https://irislab.tech/course_robotics/lec4-fk/fk.html. Accessed: May 14, 2026.
- [22] Franka Robotics, “Franka Research 3.” [Online]. Available: <https://franka.de/franka-research-3>. Accessed: May 14, 2026.
- [23] Franka Robotics, “Franka Research 3 Datasheet,” Release Version 1.2, Document no. R02212, Apr. 2024. [Online]. Available: <https://static.generation-robotics.com/media/franka-robotics-fr3-datasheet.pdf>. Accessed: May 14, 2026.
- [24] Franka Robotics, “Control Interface Specification and Robot Limits,” *Franka Control Interface Documentation*. [Online]. Available: https://frankarobotics.github.io/docs/robot_specifications.html. Accessed: May 14, 2026.
- [25] TMC Robotics, “Fairino FR3 Cobot—3 kg Payload, 622 mm Reach.” [Online]. Available: <https://www.tmc-robotics.com/product/fr3-collaborative-robot>. Accessed: May 14, 2026.
- [26] FAIRINO Europe, “FAIRINO FR3 Collaborative Robot.” [Online]. Available: <https://fairino.info/>. Accessed: May 14, 2026.